Classification and Evaluation Metrics

Lesson 4 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

30 October 2026 · reading time about 80 minutes

Contents

Lesson plan . . . 2
1. From a number to a decision . . . 2
    1.1 The dataset, and why it is synthetic . . . 3
2. The model: from a line to a probability . . . 4
    2.1 What goes wrong with a straight line . . . 4
    2.2 The sigmoid . . . 4
    2.3 Odds and log-odds: why this particular curve . . . 7
    2.4 Reading the coefficients . . . 11
3. Cross-entropy, derived from maximum likelihood . . . 14
    3.1 The idea before the algebra . . . 14
    3.2 The likelihood . . . 16
    3.3 Why we take the logarithm . . . 16
    3.4 The gradient, and a small miracle . . . 17
4. Why not squared error . . . 18
    4.1 The cost of being confidently wrong . . . 18
    4.2 The sharper problem: the gradient dies . . . 19
    4.3 Convexity . . . 20
5. The trap: accuracy . . . 21
    5.1 The number to remember from this lesson . . . 21
    5.2 Why this keeps happening . . . 22
6. The confusion matrix and what is built on it . . . 22
    6.1 Four numbers . . . 22
    6.2 Precision and recall . . . 23
    6.3 F1, and why the harmonic mean . . . 24
    6.4 Averaging across classes . . . 26
7. The threshold is a decision . . . 26
    7.1 Nothing about the model changes along this axis . . . 26
    7.2 Choosing the threshold from costs . . . 30
8. ROC and AUC . . . 33
    8.1 The curve . . . 33
    8.2 What the area means . . . 35
    8.3 Where ROC misleads . . . 38
9. Class weights and resampling . . . 39

10. More than two classes (optional) . . . 41
10.1 Softmax . . . 41
10.2 Metrics with three classes . . . 42
11. Summary . . . 44
Homework . . . 44
Notation used in this lesson . . . 44

Lesson plan

Time	Segment	Material
0:00–0:12	Exercise 3 discussed; from numbers to decisions	Slides 2–6
0:12–0:30	Logistic regression: sigmoid, odds, coefficients	Slides 7–16
0:30–0:53	Why the cost had to change, in steps	Slides 17–27
0:53–1:13	The decision boundary, and **Notebook 1** live	Slides 28–29
1:13–1:23	**Break**	Slide 30
1:23–1:37	The accuracy trap and the confusion matrix	Slides 31–34
1:37–1:51	Precision, recall, F1, and what to report	Slides 35–38
1:51–2:16	**Notebook 2**, live, and the threshold sweep	Slides 39–42
2:16–2:30	Receiver operating characteristic (ROC) curves, the area under them, and where they mislead	Slides 43–50
2:30–2:40	Cost-optimal thresholds and class weights	Slides 51–56
2:40–2:55	**Notebook 3**, live, with the optional multiclass slides 58–60	Slide 57
2:55–3:00	What to take away; homework	Slides 61–62
	Total	**180 minutes**

Taught over two Fridays in 2026: slides 1 to 42 on 30 October, slides 43 to 62 at the start of 6 November, before lesson 5.

---

1. From a number to a decision

Lesson 3 predicted a quantity: given a house, how many euros. This lesson predicts a **category**: given a disk drive, will it fail in the next thirty days?

The change of question forces two changes of machinery, and it is worth being clear at the outset that they are separate.

The model has to change, because a straight line is unbounded and a probability is not. Section 2 fixes that with the sigmoid.

The evaluation has to change, because "how far off were we, on average" is meaningless when the answer is yes or no. Sections 5 to 8 are about that, and they occupy more of this lesson than the model does. That proportion is deliberate: fitting a classifier takes one line of scikit-learn, and knowing whether it is any good is where the actual difficulty lives.

1.1 The dataset, and why it is synthetic

Everything below uses 8,000 disk drives from a data centre. Each drive reports six SMART counters — **Self-Monitoring, Analysis and Reporting Technology**, the diagnostic statistics a drive keeps about itself — and a label saying whether it failed within thirty days. The six counters, with the value of a typical drive and the range that holds the middle 90% of the fleet:

Column	What it records	Typical drive (median)	Middle 90% of drives
`reallocated_sectors`	bad patches of the disk the drive has retired and replaced with spare ones	2	0 to 6
`spin_retry_count`	times the motor needed a second attempt to bring the platters up to speed	0 (84% of drives have none)	0 to 1
`read_error_rate`	share of reads that hit an error	0.17%	0.04% to 0.70%
`seek_error_rate`	share of head movements that missed their track	0.22%	0.05% to 0.97%
`temperature_c`	operating temperature, in degrees Celsius	35.2	28.3 to 42.3
`power_on_hours`	hours in service	16,100 (about 22 months)	5,100 to 37,000

The first four are counts and rates that span orders of magnitude, which is why they are read on a logarithmic scale (below); temperature and hours are used as they are.

306 of the 8,000 drives failed: 306 / 8,000 = 3.8%.

The data is generated by `Notebooks/disk_data.py`, and we know the coefficients that produced the labels. That is the same choice as lesson 3, for the same reason: a real dataset lets you admire an estimate, and a synthetic one lets you check it. One column, `seek_error_rate`, was generated with a coefficient of **exactly zero** — a plausible-looking counter with no connection to failure. It is there to be found out.

The counters are read on a logarithmic scale. Before the labels are generated, `disk_data.py` applies $\log(1 + x)$ to the four count columns, so the truth is linear in $\log(1 + x)$ and not in x; the 1 keeps a drive with no reallocated sectors at $\log 1 = 0$ rather than $\log 0 = -\infty$. That is not a convenience: it is how the evidence behaves. Going from 0 to 4 reallocated sectors is a **17 times** larger move than going from 40 to 44 ($\log 5 = 1.61$ against $\log(45/41) = 0.093$, and $1.61/0.093 = 17$),

because the first sector a drive ever reallocates says something and the forty-first says almost nothing new. Section 2.2 draws what this does to the model.

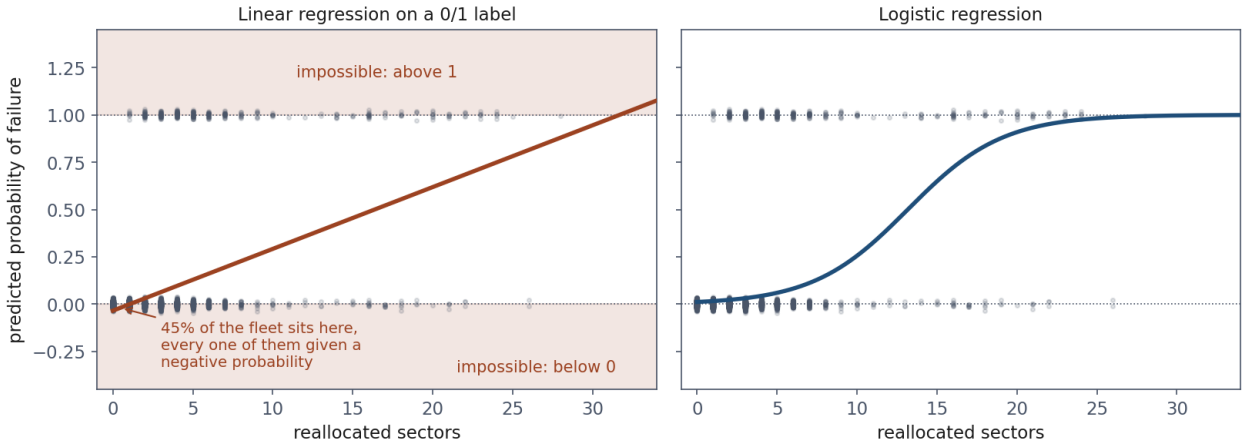
This is the feature engineering of lesson 2, and it has two consequences here. The coefficients in section 2.4 are per standard deviation of the *transformed* column. And the single-feature curve in section 2.1 is fitted on the raw count instead, so it gives every sector the same weight; section 2.2 shows what that costs.

---

2. The model: from a line to a probability

2.1 What goes wrong with a straight line

Fit ordinary least squares to the 0/1 label using one feature, the count of reallocated sectors. Nothing prevents it. The result is



Left, ordinary least squares on a 0/1 label: the shaded bands are where the line claims something that cannot be true, and 45% of the fleet (3,606 of the 8,000 drives) sits in the lower one. Right, the same data with a curve that cannot leave the interval.

$$\hat{y} = 0.0326\,x - 0.0342$$

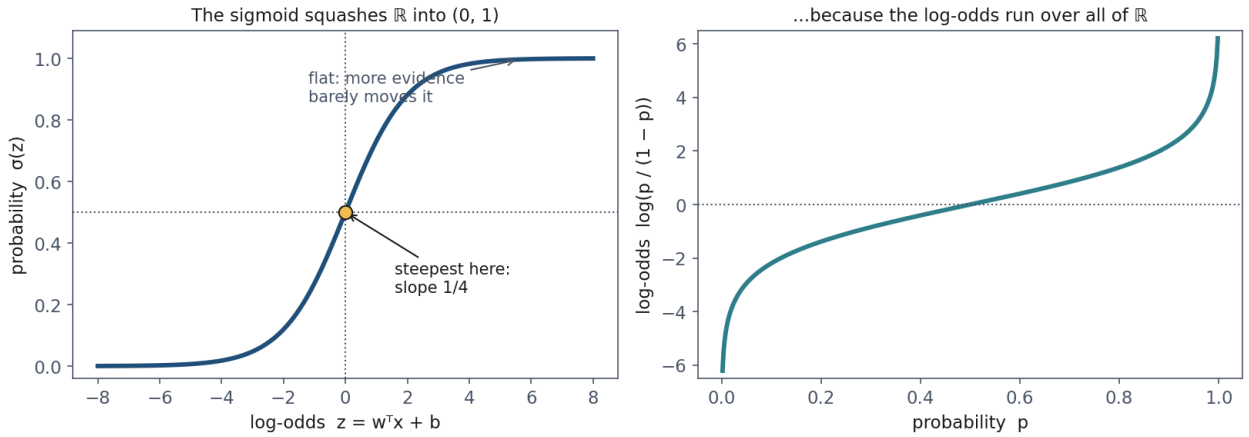
and it assigns **a negative probability to 3,606 of the 8,000 drives** — 3,606 / 8,000 = 45% of the fleet. Extend it slightly past the observed range and it predicts a probability above 1: setting $0.0326\,x - 0.0342 = 1$ gives $x = 1.0342/0.0326 = 31.7$ reallocated sectors.

This is not a defect that better fitting would remove. A straight line is unbounded and a probability lives in $[0, 1]$; no choice of w and b reconciles the two. **The shape of the function has to change, not its parameters.**

2.2 The sigmoid

The picture first. We want a function that takes any real number — the output of a linear model, which can be anything — and returns something between 0 and 1. It should be **monotone**, so more evidence never lowers the probability. And it should **flatten at both ends**, because evidence

has diminishing returns: the same increment of evidence should count for a great deal while the question is still open, and for almost nothing once it is settled.



Left, the sigmoid: steep where the evidence is genuinely ambiguous, flat where your mind is already made up. Right, the same relationship inverted — the log-odds run over the whole real line, which is exactly the range a linear model can safely predict.

The function with those properties is the **sigmoid** or **logistic function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The model becomes

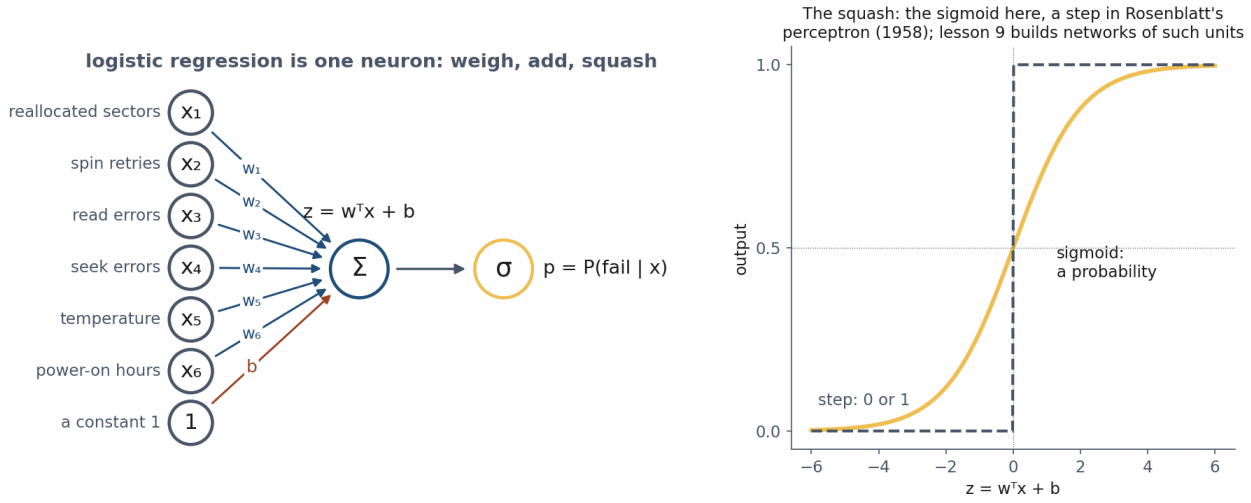
$$\hat{y} = P(y = 1 \mid x) = \sigma(w^\top x + b)$$

What p is. $\hat{y} = P(y = 1 \mid x)$, also written p, is the probability that this drive **fails within thirty days, given its counters** — a precise quantity, not a vaguer "probability of trouble". And although $w^\top x + b$ has the same form as lesson 3's line, w and b are not fitted by least squares: section 3 shows they are chosen by maximum likelihood.

What x is, from here on. In the one-feature picture of section 2.1, x was the raw count of reallocated sectors. In the model used from section 2.4 on, x holds a drive's six counters after two steps: the four counts and rates go through $\log(1 + x)$ (section 1.1), and then every column is **standardised** — `StandardScaler` subtracts the fleet's mean and divides by its standard deviation, so each feature has mean 0 and standard deviation 1. Two consequences follow. A coefficient w_j is the change in z for **one standard deviation** more of feature j (section 2.4). And $x = 0$ **is not a drive with no sectors and no errors: it is a drive at the fleet average on every counter**, so the intercept b, the value of z when every x_j is 0, is the log-odds of an average drive. Reading $x = 0$ as "no sectors, no errors" is the natural mistake, and it makes the intercept look like the risk of a perfect drive.

The model as one neuron. Drawn rather than written, the model weighs each counter, adds the products together with b, and passes the sum through the sigmoid. The diagram is older than logistic regression's use in machine learning: in 1958 Frank Rosenblatt published the **perceptron**, the same weigh-and-add followed by a hard step, so its output was a label, 0 or 1, never a probability.

It was also trained differently, by a rule that adjusts the weights only after a mistake, where logistic regression minimises log loss (section 3). **Logistic regression is one neuron**, and the neural networks of lesson 9 are many such units, side by side and one after another.



Left, logistic regression as a unit: the six counters weighted, added with b (the weight on a constant input of 1), squashed into p. Right, the squash: the sigmoid used here against the step of Rosenblatt's perceptron. Both switch at $z = 0$; only the sigmoid says how sure it is.

A predictable mistake: "the perceptron *is* logistic regression". Same diagram, different squash, different training. The step's slope is 0 everywhere except at $z = 0$, where it has none, so it gives gradient descent nothing to follow; the smooth sigmoid does.

Three properties, each of which we will use:

$$\sigma(0) = \tfrac{1}{2}, \qquad \sigma(-z) = 1 - \sigma(z), \qquad \sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

The derivative is the one that matters most. It is largest at $z = 0$, where $\sigma' = \sigma(0)\,(1 - \sigma(0)) = \tfrac{1}{2} \cdot \tfrac{1}{2} = \tfrac{1}{4}$, and it decays to zero at both ends. Section 4 shows that this is simultaneously the reason the sigmoid works and the reason squared error does not.

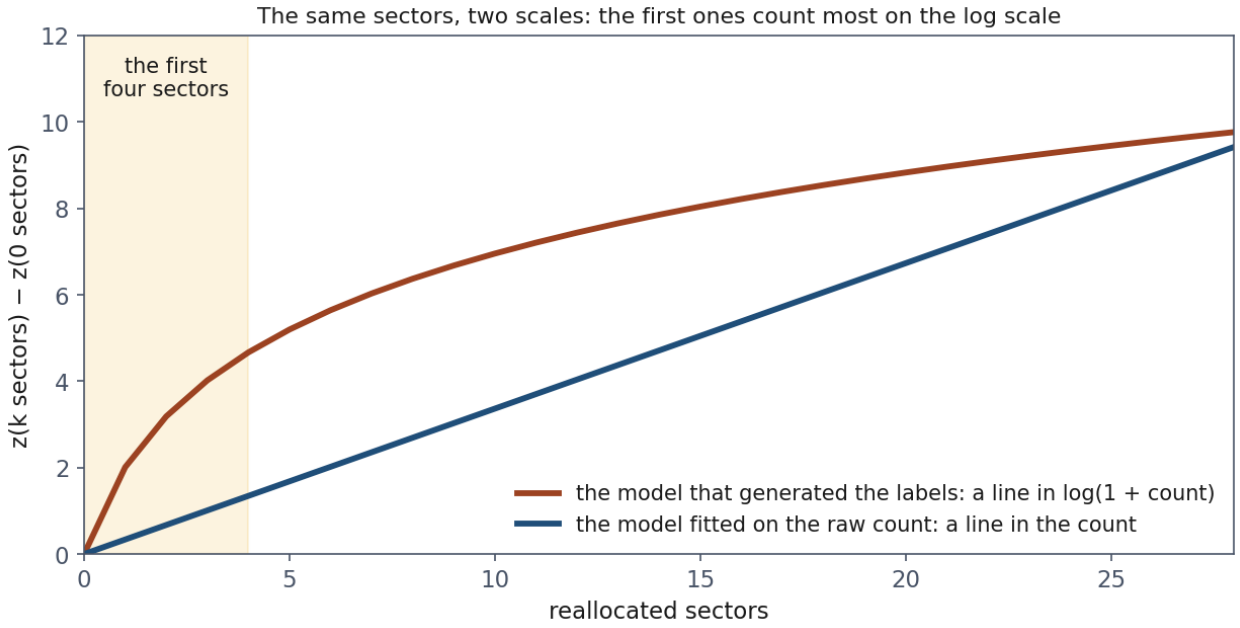
The diminishing returns, measured. They are diminishing in the *total* evidence z, not in any single counter (section 2.3 shows that z is the log-odds; for now, read it as evidence). Two units of z spent from $z = 0$ move the probability from $\sigma(0) = 0.50$ to $\sigma(2) = 1/(1 + e^{-2}) = 0.88$, a rise of 0.38; the same two units spent from $z = 6$ move it from $\sigma(6) = 0.9975$ to $\sigma(8) = 0.9997$, a rise of 0.0021 — 0.38/0.0021, **178 times less**. The curve is steep where the model is genuinely undecided and flat where its mind is made up.

A predictable mistake, and the figure invites it. Read that against the right-hand panel of the figure in section 2.1 and the two seem to disagree. That curve barely moves over the first few reallocated sectors, where 90% of the drives sit (four sectors or fewer), and it is steepest at **13.2 sectors**. It looks as if the first sectors say little. Two separate things are going on.

It is the same sigmoid, drawn against a different axis. That panel's model has a single input, the raw count x of reallocated sectors, so its z is a straight line in the count, with the slope and intercept that logistic regression fits on the raw count in notebook 1: $z = 0.336\,x - 4.43$. With no sectors,

z is already -4.43, deep in the flat left end of the curve above; it reaches 0, the steep middle, at $4.43/0.336 = 13.2$ sectors. The flat start of that panel is the flat left end of this one, with sectors on the axis instead of z.

The flat start is a fault of the model, not a fact about the drives. A drive with a few reallocated sectors is not safe; this model cannot see it, because it is a straight line in the count and gives every sector the same weight: the first sector adds as much to z as the twentieth. The labels were generated from $\log(1 + x)$ (section 1.1), which gives the first sectors the most weight. The figure shows what the sectors add to z in each case, counted from a drive with none: for two drives identical except in the sector counter, one with k sectors and one with none, the difference $z(k) - z(0)$. Everything else the two share cancels, so both curves start at 0.



What the reallocated sectors add to z, counted from a drive with none: $z(k) - z(0)$. Look at the shaded band: over the first four sectors the generating model (rust) climbs steeply, while the raw-count model (blue) climbs at the same slow rate everywhere.

That is why the model fitted from section 2.4 on uses $\log(1 + x)$.

Deriving the third property, since it is used repeatedly. Write $\sigma = (1 + e^{-z})^{-1}$ and differentiate:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\,(1 - \sigma(z))$$

using $\dfrac{e^{-z}}{1 + e^{-z}} = 1 - \dfrac{1}{1 + e^{-z}}$ in the last step.

2.3 Odds and log-odds: why this particular curve

The sigmoid is not an arbitrary S-shape. This section shows where it comes from, starting from what we want a linear model to do.

What we want, and why probability cannot give it. A linear model adds one contribution per feature, and each contribution is the same whatever the other features say. For the drives that means: the same increase in reallocated sectors moves the risk by the same amount, whatever the drive's other counters say. On the probability scale no fixed amount can do that. Say one standard deviation more of reallocated sectors adds 0.10 to the probability of failing: a drive at 50% goes to 60%, but a drive at 95% goes to 105%, which is not a probability. Any fixed step, however small, leaves $[0, 1]$ for some drive.

Evidence multiplies the odds. The **odds** are the probability that something happens divided by the probability that it does not, $p/(1-p)$: a probability of 0.2 is odds of $0.2/0.8 = 0.25$, and a probability of 0.5 is odds of $0.5/0.5 = 1$. Bayes' rule, written with odds, says how a piece of evidence changes them. Here the evidence is a **reading**: a value that one of the drive's counters reports, such as a high count of reallocated sectors — the x in the formula.

$$\frac{P(y = 1 \mid x)}{P(y = 0 \mid x)} = \frac{P(y = 1)}{P(y = 0)} \cdot \frac{P(x \mid y = 1)}{P(x \mid y = 0)}$$

The odds after seeing x are the odds before, times the **likelihood ratio**: how much more common the reading x is among drives that fail than among drives that do not. A reading six times more common among failing drives multiplies the odds by 6, whatever they were.

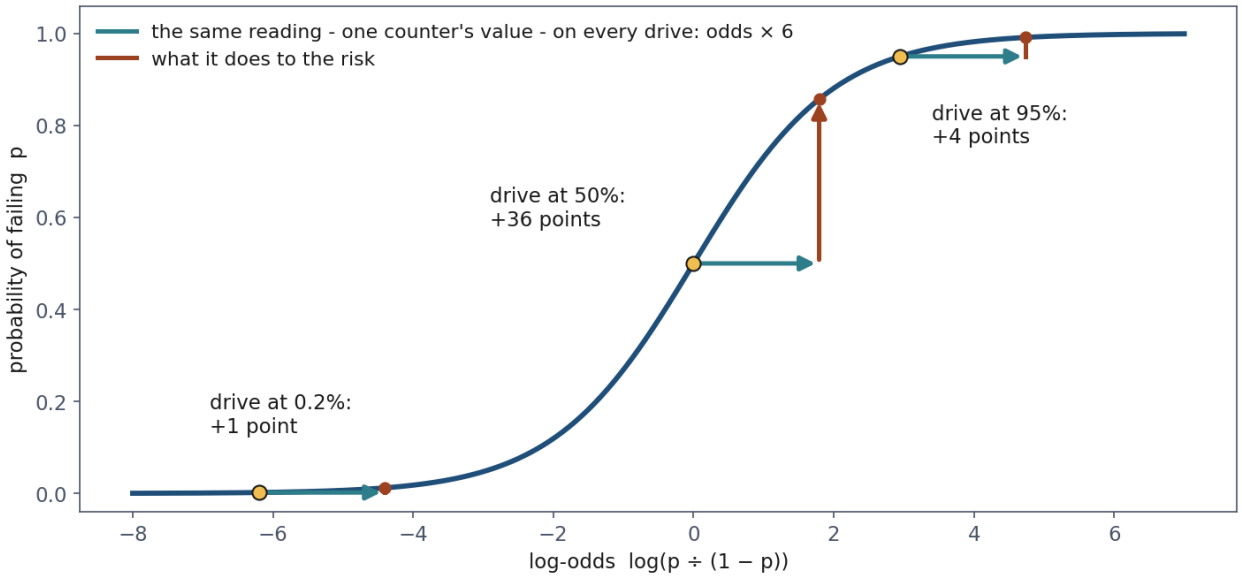
Counting drives shows why. Take 1,000 drives at 50% risk and 1,000 at 95%, and a reading that shows up on 60% of the drives that are going to fail and on 10% of the healthy ones — six times more often:

One reading, 6 times more common among failing drives	Drives at 50% risk	Drives at 95% risk
1,000 such drives: will fail / stay healthy	500 / 500	950 / 50
Those showing the reading: 60% of failing, 10% of healthy	300 / 50	570 / 5
Risk after the reading: failing ÷ all showing it	$300 \div 350 = 86\%$	$570 \div 575 = 99.1\%$
Adding 0.10 to the risk instead	60%	105%: impossible
Odds = failing ÷ healthy, before and after	1, then $300 \div 50 = 6$: $\times\ 6$	19, then $570 \div 5 = 114$: $\times\ 6$

The reading scales the failing drives by 0.6 and the healthy ones by 0.1, so their ratio — the odds — is multiplied by $0.6/0.1 = 6$ whatever it was before; the shares 60% and 10% are an illustration, and any pair with ratio 6 gives the same risks. The risk after is a count out of a total, so it can never pass 100%. A factor, unlike a step, never breaks anything — any positive odds times 6 are still positive odds, and so still a probability. The same arithmetic, written for any starting risk and including an average drive:

Risk before, p	Odds before, $p/(1-p)$	A reading 6 times more common among failing drives: odds × 6	Risk after, odds /(1+ odds)	Adding 0.10 to the risk instead
0.2% (an average drive: $\sigma(-6.20)$, the true intercept)	0.002	0.012	1.2%	10.2%
50%	1	6	86%	60%
95%	19	114	99.1%	105%: impossible

The factor 6 is realistic for this fleet: the coefficient of reallocated sectors used to generate the labels is 1.80 per standard deviation, a factor of $e^{1.80} = 6.05$ on the odds (section 2.4 reads coefficients this way). Read the "Risk after" column. The same evidence adds $86 - 50 = 36$ points to the probability at 50%, but $1.2 - 0.2 = 1$ point at 0.2% and $99.1 - 95 = 4$ points at 95%. That is the shape of section 2.2 — steep in the middle, flat at both ends — and nothing chose it: it is what multiplying the odds looks like on the probability scale.



The same reading on three drives. Look at the arrows: the teal ones, the reading multiplying the odds by 6, all have the same length on the log-odds axis; the rust ones, what it does to the probability, are long in the middle (+36 points at 50%) and short at both ends (+1 point at 0.2%, +4 at 95%).

The logarithm turns factors into steps. Several readings, if they are independent given the class, multiply the odds by one factor each. Take the logarithm and the factors become terms that add:

$$\log(\text{odds after}) = \log(\text{odds before}) + \log(\text{factor}_1) + \log(\text{factor}_2) + \ldots$$

A starting value plus one term per feature is exactly the form of a linear model, $w^\top x + b$. The starting value in Bayes' rule is the log-odds before any reading, from the base rate alone: for

this fleet $\log(306/7694) = -3.22$, a probability of 3.8%. That is not the intercept. Each factor's logarithm depends on its reading, and once the features are centred (standardised, as here) the constant part of every such term is folded, with the base rate, into one number b: the log-odds at $x = 0$, the drive with average telemetry (-6.20, and $\sigma(-6.20) = 0.2\%$, the first row above). Each remaining term $w_j x_j$ is the logarithm of the factor that feature multiplies the odds by, relative to an average reading. That is why the model puts the line on the **log-odds**, and only then turns it back into a probability. Logistic regression assumes, in addition, that each feature's term is a straight line in that feature.

A predictable mistake: "the intercept is the risk before looking at the counters". The derivation starts from "the odds before" and ends at b, so the reading is natural. But the risk of a drive drawn at random from the fleet is 3.8% (log-odds -3.22), while b is the log-odds of the average drive, 0.2%. The two differ because the failures come from the drives with bad telemetry, not from the average one.

And a second: "logistic regression assumes the counters are independent". Independence given the class was used above only to motivate why evidence adds on the log-odds scale. Fitting logistic regression does not require it: correlated counters share the weight through their fitted coefficients, which is one of its advantages over naive Bayes, which does assume it.

The log-odds also has the range the line needs: the odds have no ceiling, as the probability approaches 1 they run off to infinity, and the logarithm removes their floor at 0, so the log-odds runs over the whole real line. But that range is a consequence, not the reason; the reason is the way evidence combines. Setting $p = \sigma(z)$ and solving for z:

$$z = \log \frac{p}{1-p}$$

so

$$\log \frac{P(y=1 \mid x)}{P(y=0 \mid x)} = w^\top x + b$$

Logistic regression is a linear model of the log-odds. The sigmoid is not a squashing trick bolted onto linear regression; it is what you get when you model the log-odds linearly and then ask for the probability back.

The same algebra, run forwards. The lecture takes it in the other order: decide first that the log-odds is linear, $\log \frac{p}{1-p} = z$ with $z = w^\top x + b$, and solve for p. Exponentiate both sides, $\frac{p}{1-p} = e^z$; multiply out and collect p, $p\,(1 + e^z) = e^z$; then divide top and bottom by e^z:

$$p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z)$$

So the sigmoid is not chosen first and justified afterwards. It is the solution of that equation.

Probability p	Odds, $p/(1-p)$	Log-odds, $\log \frac{p}{1-p}$
0.01	1 to 99	-4.60
0.10	1 to 9	-2.20
0.50	1 to 1	0

Probability p	Odds, $p/(1-p)$	Log-odds, $\log \frac{p}{1-p}$
0.90	9 to 1	+2.20
0.99	99 to 1	+4.60

Why the log-odds, and not some other stretch. The log-odds is not the only function that maps $(0, 1)$ onto the whole real line. The **probit**, the inverse of the standard normal distribution's cumulative distribution function, is the common alternative, and once rescaled it draws almost the same S-shape. So the shape is not what singles the sigmoid out. Three properties are:

- **The coefficients have a direct interpretation.** On the log-odds scale a one-unit increase in x_j adds w_j to the log-odds, which multiplies the odds by e^{w_j} (section 2.4). A probit coefficient has no interpretation this direct.
- **It is the scale on which evidence adds**, as the start of this section showed: Bayes' rule multiplies the odds by one factor per reading, and the logarithm turns those factors into the terms of a linear model.
- **Training is well behaved.** With the cross-entropy cost of section 3 the gradient comes out as $(p-y)\,x$, the same form as linear regression's, and the cost is convex (sections 3.4 and 4.3).

A predictable mistake, and why it is reasonable. Most students come away thinking the sigmoid was picked because it has the right shape. Section 2.2 invites that interpretation, since it introduced the curve by its properties: bounded, monotone, flat at both ends. But the probit has every one of those properties too. What makes the sigmoid the standard choice is the log-odds behind it.

2.4 Reading the coefficients

Because z is log-odds, e^{w_j} is a **multiplier on the odds** for a one-unit increase in x_j. With standardised features, one unit is one standard deviation — of the transformed column, for the four counters that section 1.1 puts on a logarithmic scale.

The **coefficients** are the w_j of the model, one per feature; machine learning also calls them **weights**, and scikit-learn stores them in `coef_`. The constant b is the **intercept**, stored in `intercept_` (neural networks call it the bias). The *class weights* of section 9 are a different thing: how much each training example counts in the cost, not a w_j.

For the four counters on a logarithmic scale, one standard deviation is not "one more", and it takes four steps to see what it is.

Step 1: what the model sees. Not the count x of reallocated sectors, but $\log(1+x)$, standardised. So the counter contributes to z

$$1.80 \times \frac{\log(1+x) - 0.96}{0.62}$$

where 0.96 and 0.62 are the mean and the standard deviation of $\log(1+x)$ over the fleet, and 1.80 is the true coefficient.

Step 2: one standard deviation more is 0.62 more on $\log(1+x)$ — not one sector more. The fraction above grows by exactly 1, and z by 1.80.

Step 3: adding 0.62 to a logarithm multiplies what is inside it by $e^{0.62} = 1.86$, because $\log(1+x) + 0.62 = \log\left((1+x) \times e^{0.62}\right)$. In sectors:

Sectors now, x	$1+x$	$(1+x) \times 1.86$	Sectors after one SD more
1	2	3.7	2.7
4	5	9.3	8.3
10	11	20.5	19.5

Roughly, the count doubles, and each such step multiplies the odds by the same $e^{1.80} = 6.05$: going from 1 sector to 2.7 counts as much as going from 10 to 19.5.

Step 4: so one extra sector is worth a lot at the start and little later. Going from x to $x+1$ adds $1.80 \times \log\left((x+2)/(x+1)\right)/0.62$ to z, and multiplies the odds by e to that:

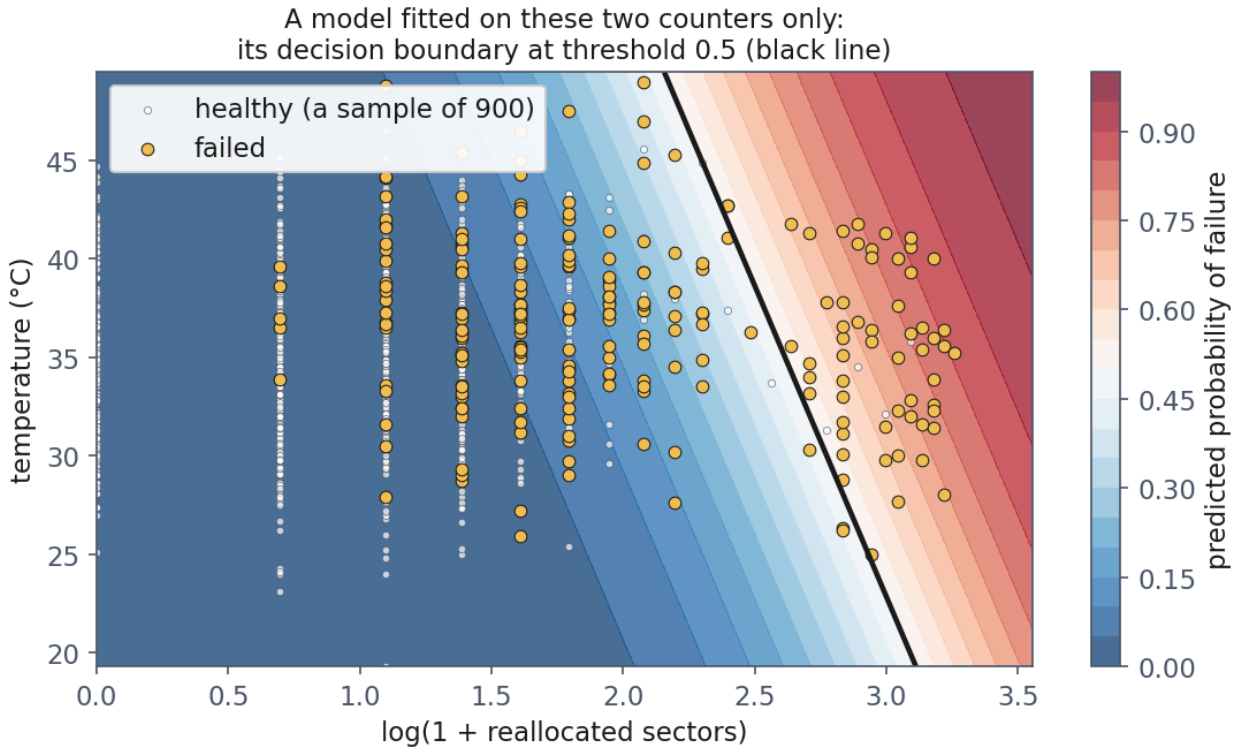
One extra sector	Added to $\log(1+x)$	Added to z	Odds multiplied by
$0 \to 1$	$\log(2/1) = 0.69$	2.01	7.5
$1 \to 2$	$\log(3/2) = 0.41$	1.18	3.2
$4 \to 5$	$\log(6/5) = 0.18$	0.53	1.7
$10 \to 11$	$\log(12/11) = 0.087$	0.25	1.3

This is why the first sectors weigh so much, and why section 1.1 put the counters on a logarithmic scale: the labels were generated that way. Temperature and hours are standardised on their own values, so for them one standard deviation is an ordinary additive step.

The reason takes two lines. Raising x_j by one unit, with every other feature held where it is, adds exactly w_j to $z = w^\top x + b$. And since z is the logarithm of the odds, the odds are e^z, so

$$\text{odds after} = e^{z+w_j} = e^z \cdot e^{w_j} = \text{odds before} \cdot e^{w_j}$$

whatever z was. A sum on the log-odds is a product on the odds: the logarithm turns products into sums, and this is the same fact read backwards. On an average drive, with the true coefficients: its log-odds are the intercept, -6.20, so its odds are $e^{-6.20} = 0.0020$. One standard deviation more of reallocated sectors adds 1.80, giving log-odds of -4.40 and odds of $e^{-4.40} = 0.0123$ — six times as large, because $e^{1.80} = 6.05$. That is the teal step of the figure in section 2.3. Back to a probability, with $p = \text{odds}/(1 + \text{odds})$: from $0.0020/1.0020 = 0.2\%$ to $0.0123/1.0123 = 1.2\%$.



Two features, so the model can be drawn. The boundary is a straight line — logistic regression is a linear classifier, and the curve is in the probabilities rather than in the boundary. Note how many gold points sit on the healthy side: that is the threshold, not the model.

Our fitted model gives, against the truth used to generate the data:

Feature	True w_j	Fitted (notebook 1)	Odds multiplier, e^{fitted}
reallocated_sectors	1.80	1.66	5.28
spin_retry_count	1.15	1.17	3.22
read_error_rate	0.85	0.87	2.39
temperature_c	0.45	0.46	1.59
power_on_hours	0.35	0.38	1.46
seek_error_rate	**0.00**	**0.02**	**1.02**
intercept	−6.20	−6.09	—

Two things to read off it.

The decoy was caught. `seek_error_rate` was generated with no effect, and the model estimates it at 0.02, an odds multiplier of $e^{0.02} = 1.02$. It did not fall for a plausible-looking column. The harder question — would you have noticed without being told? — is what lesson 5 is about.

The intercept is the average drive's chance, not the base rate. A drive with perfectly average telemetry has every standardised feature at 0 — at 0 standard deviations from the fleet's mean, not at zero sectors (section 2.2) — so its log-odds are the intercept alone, and by the sigmoid $\sigma(-6.09) = 1/(1 + e^{6.09}) = 1/442 = 0.0023$: a 0.2% chance of failing in thirty days, while the fleet as a whole fails at 3.8% (section 1.1). The gap is the point: failures come from the small group of

drives far above average on one or more counters, not from a typical drive. To reach an even bet, $p = 0.5$, the log-odds must reach $\log(0.5/0.5) = 0$, so the counters must add more than 6.09 to the intercept's -6.09, and only badly degraded drives do that. **That single number explains why every metric in section 5 behaves the way it does.**

A predictable mistake, and the reasonable thinking behind it. Most students read "odds multiplier 5.28" as "five times as likely to fail", and the instinct is sound — for rare events the odds and the probability are nearly the same number, so that interpretation is *almost* right here. It stops being right as probabilities grow: a probability of 0.90 is odds of $0.9/0.1 = 9$; multiplied by 5 that gives odds of 45, and turning odds back into a probability, $p = \text{odds}/(1 + \text{odds}) = 45/46 = 0.978$. The odds went up fivefold; the probability moved by under eight points. **Odds ratios multiply odds, not probabilities.**

---

3. Cross-entropy, derived from maximum likelihood

3.1 The idea before the algebra

The picture first. We have a model with a knob — the parameters — and each setting of the knob assigns a probability to every drive failing or surviving. Some settings assign high probability to what actually happened; others assign low probability to it. **Maximum likelihood says: turn the knob to whichever setting makes the data we actually observed as unsurprising as possible.**

That is the whole idea. The algebra below turns "as unsurprising as possible" into a cost function, and the pleasant surprise is that the cost function it produces is not something anyone had to invent — it falls out.

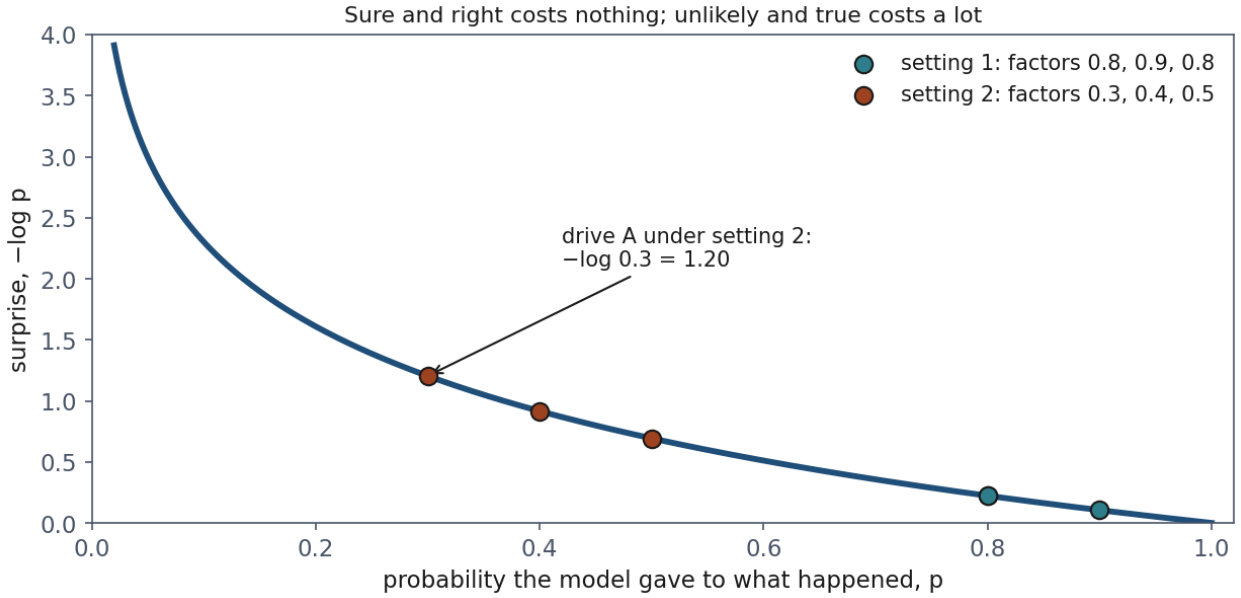
Three drives, two settings. Take drive A, which failed, and drives B and C, which did not, and two settings of (w, b). Setting 1 gives them failure probabilities 0.8, 0.1 and 0.2; setting 2 gives 0.3, 0.6 and 0.5. The numbers are made up for the illustration. Score each setting by the probability it gave to what actually happened, p for the drive that failed and $1-p$ for the two that did not. Then ask how probable it was that **all three** outcomes happened together. If the drives fail or survive independently of one another, the probability that several things all happen is the product of their probabilities — the same rule that gives two heads from two fair coins a probability of $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. So multiply:

Drive: what happened	Setting 1: probability of what happened	Setting 2: probability of what happened
A: failed	$p = 0.8$	$p = 0.3$
B: stayed healthy	$1 - p = 1 - 0.1 = 0.9$	$1 - p = 1 - 0.6 = 0.4$
C: stayed healthy	$1 - p = 1 - 0.2 = 0.8$	$1 - p = 1 - 0.5 = 0.5$
All three, multiplied: the likelihood	$0.8 \times 0.9 \times 0.8 = \mathbf{0.576}$	$0.3 \times 0.4 \times 0.5 = \mathbf{0.060}$

A predictable mistake. The middle columns are not the probability of failing: they are the probability of **what happened** — of failing for A, of staying healthy for B and C. The model's p is the probability of failing for every drive, so for B and C the factor is $1 - p$.

What actually happened is $0.576/0.060 = 9.6$ times more probable under setting 1, so maximum likelihood prefers it. Note what the comparison does and does not say: 0.576 on its own has no scale, and a likelihood is only useful against another setting; and setting 1 is not *the* maximum — it merely beats setting 2. Fitting the model is the same comparison made over every possible setting, and the winner of that is the maximum likelihood.

The last step turns the product into a cost, in two moves with a reason each (section 3.3 has them in full). **The logarithm** does not change the winner — it is increasing, so the larger likelihood keeps the larger log — and it turns the product into a sum, one term per drive, which can be differentiated term by term and, unlike a product of 6,000 factors, does not underflow to zero. **The minus** makes every term positive, since the log of a probability is at most 0, and turns "largest" into "smallest", so the best setting is the one with the smallest cost. The logarithm is a convenience, not a new assumption: the best setting is the same with it or without it. Taking minus the (natural) logarithm of each factor and adding gives, for setting 1, $-\log 0.8 - \log 0.9 - \log 0.8 = 0.22 + 0.11 + 0.22 = 0.55$, and for setting 2, $-\log 0.3 - \log 0.4 - \log 0.5 = 1.20 + 0.92 + 0.69 = 2.81$. The log of a product is the sum of the logs, so each total is minus the log of that setting's likelihood: the same order reversed, so the better setting now has the smaller number, which can be minimised as in lesson 3. Minus the log of a probability is the *surprise* of the outcome, which is why "most likely" and "least surprising" are the same choice. The name fits what $-\log p$ does: it is 0 for an outcome the model was certain of, grows as the outcome was called less likely, is infinite for one called impossible, and adds up over independent outcomes, since the log of a product is the sum of the logs. (The term comes from information theory and is not examinable.) Section 3.3 arrives at the same quantity, averaged over the drives, as the cost.



The surprise of an outcome, $-\log p$, against the probability p the model gave it. Look at where the six factors above land: setting 1's (teal) all sit on the flat right-hand part, setting 2's (rust) further up, and A's 0.3 highest of all.

3.2 The likelihood

Write $p^{(i)} = \sigma(w^\top x^{(i)} + b)$ for the model's probability that drive i fails. Each drive contributes one factor: $p^{(i)}$ if it failed, $1 - p^{(i)}$ if it did not. Both cases fit in one expression, because $y^{(i)}$ is 0 or 1 and anything to the power 0 is 1:

$$P(y^{(i)} \mid x^{(i)}) = \left(p^{(i)}\right)^{y^{(i)}} \left(1 - p^{(i)}\right)^{1 - y^{(i)}}$$

Assuming the drives are independent given their telemetry, the probability of the whole dataset is the product:

$$\mathcal{L}(w, b) = \prod_{i=1}^{m} \left(p^{(i)}\right)^{y^{(i)}} \left(1 - p^{(i)}\right)^{1 - y^{(i)}}$$

Two words, kept apart. The first expression is one drive's **factor**: the probability the model gave to what happened to that drive. The **likelihood** is the product, read as a function of (w, b) with the data held fixed — which is why it is written $\mathcal{L}(w, b)$. Calling the single factor "the likelihood of one example" is a common slip; it blurs the fact that the likelihood scores a *setting* of the parameters against *all* the data at once.

3.3 Why we take the logarithm

Two reasons, one mathematical and one entirely practical.

Mathematically, the logarithm turns the product into a sum, and sums differentiate one term at a time. It does not move the maximum, because log is strictly increasing.

Practically, the product underflows. Notebook 1's gradient descent starts with every coefficient and the intercept at zero, where every one of the 6,000 training drives gets probability 0.5, and $0.5^{6000} = 10^{6000 \log_{10} 0.5} = 10^{6000 \times (-0.301)} = 10^{-1806}$. The smallest positive number a 64-bit float can hold is about 5×10^{-324} (and about 2×10^{-308} while it keeps its full precision), so the product is stored as exactly zero, and so is every gradient computed from it. Even the fitted model, at an average log loss of 0.0692, multiplies out to only about 10^{-180}: the log loss is minus the average log of the factors, so the product is $e^{-6000 \times 0.0692} = e^{-415} = 10^{-415/\ln 10} = 10^{-180}$. Every serious implementation works in log space for this reason, not for elegance.

Taking the logarithm and negating — so that we minimise rather than maximise, matching the convention of lesson 3 — gives the **binary cross-entropy** or **log loss**:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log p^{(i)} + \left(1 - y^{(i)}\right) \log\left(1 - p^{(i)}\right) \right]$$

The single-example loss is

$$L(p, y) = \begin{cases} -\log p & \text{if } y = 1 \\ -\log(1 - p) & \text{if } y = 0 \end{cases}$$

Read it as a sentence: **the cost of an outcome is the surprise of seeing it.** An event you assigned probability 1 costs nothing; one you assigned probability 0.001 costs $-\log 0.001 =$

$\log 1000 \approx 6.9$; one you declared impossible costs infinity. That is why log loss is unbounded, and section 4 is about why that is exactly what we want.

A predictable mistake. In the compact form of J, with $y = 1$ the second term looks like $0 \cdot \log 0$, that is $0 \cdot (-\infty)$. It is not: the coefficient is $1 - y$, but the logarithm's argument is $1 - p$, and $p = \sigma(z)$ lies strictly in $(0, 1)$, so $\log(1 - p)$ is finite and the term is 0. Only p exactly 1 would give $\log 0$, which the sigmoid never reaches. A computer can round p to 1.0 when z is large, which is why implementations clip p into $[\varepsilon, 1 - \varepsilon]$ — notebook 1 adds 10^{-12} inside the logarithm, and scikit-learn's `log_loss` clips.

3.4 The gradient, and a small miracle

Differentiate the loss for a single example with respect to $z = w^\top x + b$, by the chain rule, in four steps. Two derivatives are already known: the loss, $L = -[y \log p + (1 - y) \log(1 - p)]$, and the sigmoid's, $dp/dz = p(1 - p)$ (section 2.2).

Step 1: how L changes with p. The derivative of $\log p$ is $1/p$, that of $\log(1 - p)$ is $-1/(1 - p)$:

$$\frac{\partial L}{\partial p} = -\left[\frac{y}{p} - \frac{1 - y}{1 - p}\right]$$

Step 2: the chain rule, $\partial L/\partial z = (\partial L/\partial p) \times (dp/dz)$:

$$\frac{\partial L}{\partial z} = -\left[\frac{y}{p} - \frac{1 - y}{1 - p}\right] p\,(1 - p)$$

Step 3: multiply into the bracket. $p(1 - p)/p = 1 - p$ and $p(1 - p)/(1 - p) = p$, so

$$\frac{\partial L}{\partial z} = -\left[y(1 - p) - (1 - y)\,p\right] = -\left[y - yp - p + yp\right] = p - y$$

The two yp cancel: the $p(1 - p)$ from the sigmoid has been absorbed by the $1/p$ and $1/(1 - p)$ from the logarithms.

Step 4: the two labels. For a failed drive, $y = 1$: $\partial L/\partial z = p - 1$, negative, of size $1 - p$, which is the straight line in the right panel of the figure in section 4.2. For a healthy drive, $y = 0$: $\partial L/\partial z = p$.

From one drive to the cost. $z = w^\top x + b$ changes with w_j as x_j and with b as 1, and J is the average of L over the m training drives, so

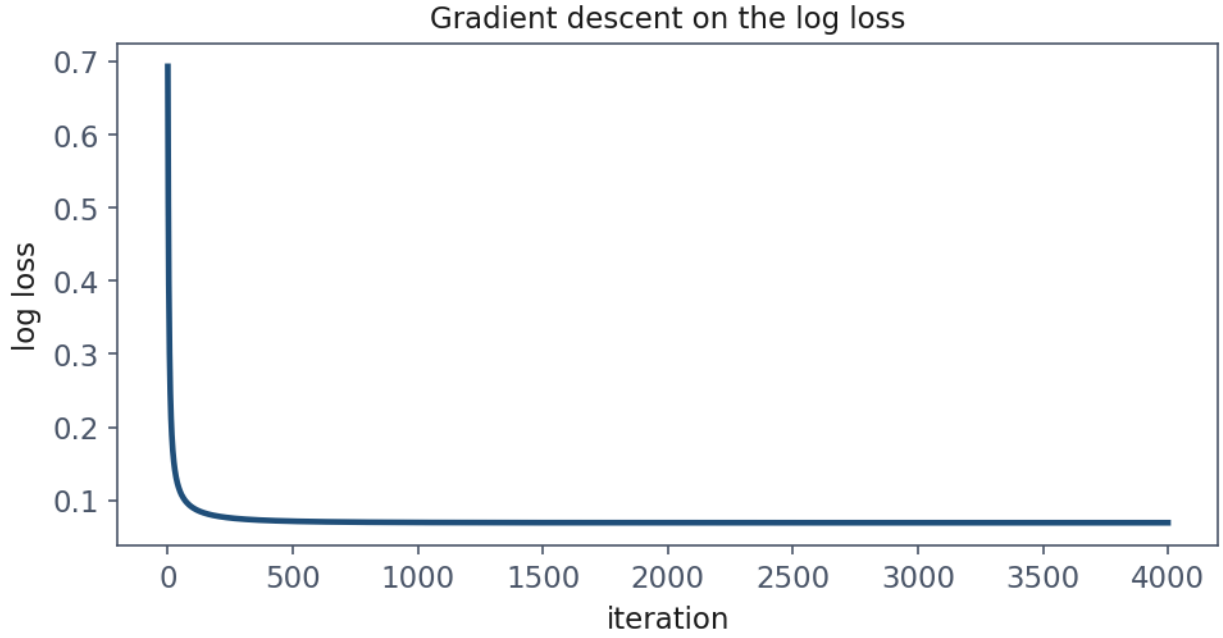
$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)} \qquad\qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)$$

The second is the first with $x_j = 1$, the constant input the intercept multiplies: the gradient with respect to b is the average error.

This is character-for-character the gradient of lesson 3. Prediction error times feature, averaged. A different model and a different cost function produce an identical update rule.

That is not a coincidence, and it is worth knowing why: both are **generalised linear models** fitted by maximum likelihood, linear regression assuming Gaussian noise and logistic regression assuming Bernoulli outcomes. For any generalised linear model with its **canonical link** (the identity for Gaussian noise, the logit for Bernoulli outcomes) the maximum-likelihood gradient takes this form. You do not need that theory to use either method — but it explains why the same gradient descent code, unchanged, trains both.

The same code, run on the drives. Notebook 1 implements that update in fifteen lines and runs it for 4,000 iterations on the six standardised features, starting from $w = 0$ and $b = 0$. The log loss falls from 0.6931 to 0.0692.



What to look at: the descent is over almost at once. The first 100 iterations cover 96.7% of the total fall — $(J_0 - J_{100})/(J_0 - J_{4000})$, with $J_0 = 0.6931$ and $J_{4000} = 0.0692$ — and the last 100 of the 4,000 change the loss by nothing at the fifth decimal — the run is long out of caution, not necessity.

The starting value is a free test. With $w = 0$ and $b = 0$ the model gives every drive $p = 0.5$, so the loss must begin at $-\log \frac{1}{2} = \log 2 = 0.6931$, which is what the notebook prints. **A from-scratch implementation that starts anywhere else has a bug in its initialisation** — one number, checked before any of the training matters.

---

4. Why not squared error

4.1 The cost of being confidently wrong

Suppose a drive failed and the model said 0.0001.

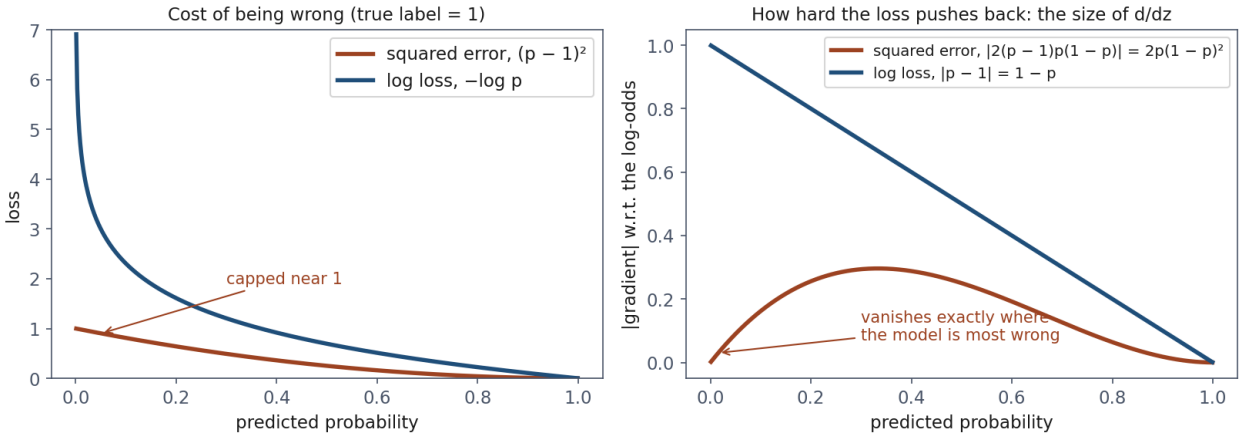
Loss	Value
Squared error $(p-1)^2$	$(0.0001-1)^2 = 0.9998$
Log loss $-\log p$	$-\log 0.0001 = 9.21$

Squared error charges **0.9998 for a confident falsehood and** $(0.5-1)^2 = 0.25$ **for an honest "don't know"**. A factor of $0.9998/0.25 = 4$ separates the worst possible answer from a shrug. Log loss charges 9.21 against $-\log 0.5 = 0.69$, a factor of $9.21/0.69 = 13$, and the ratio grows without bound as $p \to 0$.

A predictable mistake: "log loss charges more, so gradient descent works better". The size of a cost does not matter to gradient descent — multiply a cost by 100 and its minimum sits in the same place. What matters is the gradient, which section 4.2 compares: the costs only set up the question.

4.2 The sharper problem: the gradient dies

The costs matter less than the gradients, because gradients are what the optimiser uses. Differentiate both with respect to z, for an example with $y = 1$:



Left, what each loss charges for being wrong: squared error is capped near 1, log loss is not. Right, the part that matters — squared error's gradient collapses towards zero exactly where the model is most confidently wrong, while log loss pushes hardest there.

$$\frac{\partial}{\partial z}(p-1)^2 = 2(p-1)\,p\,(1-p) \qquad \frac{\partial}{\partial z}\left[-\log p\right] = p - 1$$

The squared-error gradient carries an extra factor $p(1-p)$, which is $\sigma'(z)$ — and σ' vanishes at both ends.

p	|grad|, squared error: $2(1-p)\,p\,(1-p)$	|grad|, log loss: $1-p$
0.5	0.250	0.500
0.1	0.162	0.900
0.01	0.0196	0.990
0.001	0.0020	0.999

As the model becomes more confidently wrong, squared error asks it to change less. At $p = 0.001$ the gradient is a five-hundredth of the log loss gradient: the ratio of the two is

$2p(1-p) = 2 \times 0.001 \times 0.999 = 0.002$. Log loss goes the other way: its gradient approaches its maximum exactly where the error is worst.

The cancellation in section 3.4 is the same fact seen from the other side. Log loss is *designed*, by maximum likelihood, so that the sigmoid's vanishing derivative cancels out. Squared error is not, so it does not.

A predictable mistake: "log loss's slope is $-1/p$". That is the slope of the left panel's blue curve, the derivative of $-\log p$ with respect to p. The right panel differentiates with respect to z, and there log loss's gradient is $p - 1$: size $1 - p$, a straight line from 1 down to 0. The chain rule joins the two panels, $\frac{d}{dz}\left[-\log p\right] = -\frac{1}{p} \cdot p(1-p) = -(1-p)$: the $1/p$ cancels the sigmoid's p, which is why log loss's push does not die. Squared error has no $1/p$ to cancel it.

4.3 Convexity

There is a third reason, and section 3.4 has already done most of the work for it.

J as defined in section 3.3 is **convex** in (w, b), which takes two steps to see. To keep b in the picture, append a constant 1 to every $x^{(i)}$ and treat b as one more weight; w and x below include it. Section 3.4 gave the gradient, $\partial J/\partial w = \frac{1}{m}\sum_i (p^{(i)} - y^{(i)})\, x^{(i)}$. Differentiating it once more, the only thing that depends on w is $p^{(i)}$, and by the chain rule through $z^{(i)} = w^\top x^{(i)} + b$ its derivative is $p^{(i)}(1 - p^{(i)})\, x^{(i)}$. So the matrix of second derivatives is

$$H = \frac{1}{m}\sum_{i=1}^{m} p^{(i)}\left(1 - p^{(i)}\right) x^{(i)}\left(x^{(i)}\right)^\top = \frac{1}{m} X^\top S X, \qquad S = \operatorname{diag}\left(p^{(i)}(1 - p^{(i)})\right)$$

Convexity is then one line. A matrix is positive semi-definite when $u^\top H u \geq 0$ for every direction u, and here

$$u^\top H u = \frac{1}{m}\sum_{i=1}^{m} p^{(i)}\left(1 - p^{(i)}\right)\left(\left(x^{(i)}\right)^\top u\right)^2 \geq 0$$

because every term is a positive number, $p(1-p)$, multiplying a square. There is no direction in which the surface curves downwards. Any local minimum is therefore global, and gradient descent cannot be trapped. Convexity does not promise a unique minimum, nor even a finite one: if a line separates the two classes perfectly, the cost keeps falling as the weights grow, and it is the penalty scikit-learn adds by default that keeps them finite.

Squared error composed with the sigmoid has no such guarantee, and is in general non-convex in w. Section 4.2 already shows why, for one failed drive taken as a function of z: squared error's slope $2(p-1)\,p\,(1-p)$ is 0 far to the left, where $p \to 0$ (the push that died), negative in the middle, and 0 again far to the right, where $p \to 1$. A convex function's slope can only grow as z grows; this one falls and then rises, so the curve is not convex — and since $z = wx + b$, neither is it as a function of w. **The vanishing push and the lost convexity are the same fact.** So the choice of cost function is not a matter of taste: it determines whether the optimisation problem is one we know how to solve.

A caution students reasonably get wrong. Convex does not mean there is a closed-form solution. Setting the gradient to zero gives $X^\top(\sigma(Xw + b) - y) = 0$, which is not linear in w — the sigmoid is in the way — and has no algebraic solution. There is **no normal equation for logistic**

regression. It must be solved iteratively, which is why `LogisticRegression` has a `max_iter` argument and `LinearRegression` does not.

Which iterative method scikit-learn uses. Not the gradient descent of section 3.4. The default is `solver="lbfgs"`: limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS), a quasi-Newton method. Where gradient descent takes a step of fixed size α along the gradient, L-BFGS estimates the curvature of J — its Hessian — from the last few gradients, and uses it to choose both the direction and the length of each step, so there is no learning rate to set. On notebook 1's data it stops after 15 iterations, where the from-scratch descent ran 4,000. The other solvers (`newton-cg`, `newton-cholesky`, `sag`, `saga`, `liblinear`) reach the same minimum by other routes; `saga` is the one that supports an L1 penalty, and `liblinear` penalises the intercept as well.

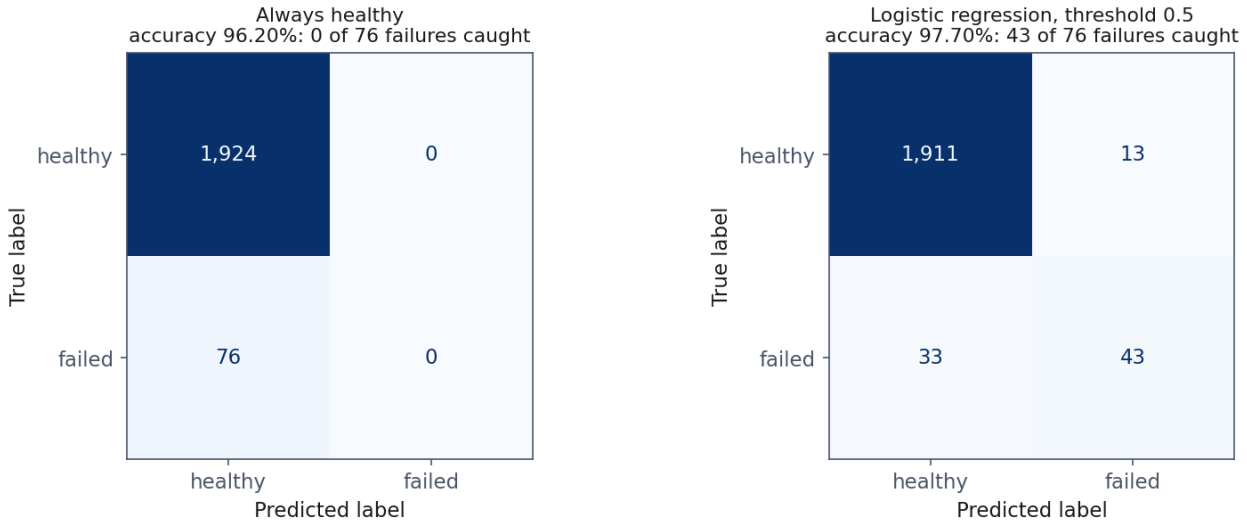
And it does not minimise J alone. The default adds an L2 penalty on the coefficients, as lesson 3's ridge did, with `C=1.0`; C is the inverse of the penalty strength, so a smaller C penalises harder. That is why notebook 1's from-scratch coefficients and scikit-learn's differ in the second decimal (1.685 against 1.664 for reallocated sectors): refitted with `penalty=None`, scikit-learn gives 1.690, and the gap closes. The default is wanted, not a bug: when a line separates the classes perfectly the unpenalised weights grow without limit, as the convexity argument above allows, and lesson 5 shows how to choose C by cross-validation.

---

5. The trap: accuracy

5.1 The number to remember from this lesson

Our test set is 2,000 drives, 76 of which failed. Consider two models.



The same test set under two models. The accuracy figures differ by 97.70 − 96.20 = 1.5 points; the number that separates them — 0 failures caught against 43 — appears in neither.

Model	Accuracy = right answers / 2,000	Failures caught
Predict "healthy" for everything	every healthy drive: 1,924 / 2,000 = **96.20%**	**0 of 76**
Logistic regression, threshold 0.5	43 failures + 1,911 healthy: 1,954 / 2,000 = 97.70%	43 of 76

96.20% accuracy, nothing caught. A model containing no learning of any kind, which could be written as `return 0`, is within $97.70 - 96.20 = 1.5$ percentage points of a model that finds more than half the failing drives (43 of 76).

The arithmetic is not subtle. 1,924 of the 2,000 drives are healthy, so answering "healthy" collects 1,924 correct answers for free: 1,924 / 2,000 = 96.20%. Only the 76 failures, 76 / 2,000 = 3.8% of the test set, are left to compete over, and no amount of skill moves accuracy far.

5.2 Why this keeps happening

This is the predictable mistake of the lesson, and the reasoning behind it is completely sound: accuracy is the fraction of predictions that were correct, and on a balanced problem whose two errors cost the same that is exactly what you want to know. It is not a naive metric. The second condition matters as much as the first: accuracy counts every error the same, so even on a balanced fleet it is the wrong summary when a miss costs more than a false alarm, as it does here (section 7.2).

It stops being informative precisely when one class is rare — which is to say, on most problems anyone builds a classifier for. Fraud, disease screening, equipment failure, intrusion detection, spam: **the interesting class is almost always the small one.** Accuracy is not wrong. It is answering a question nobody asked.

The general statement: with a positive rate π, the trivial always-negative classifier scores $1 - \pi$. At $\pi = 0.038$ that is $1 - 0.038 = 96.2\%$; at $\pi = 0.001$, as in some fraud problems, it is $1 - 0.001 = 99.9\%$.

The over-correction is predictable too: "so accuracy is useless". It is a number with a baseline. Read against $1 - \pi$ it says something; alone it never says which errors were made, which is what section 6 is for.

---

6. The confusion matrix and what is built on it

Section 5 showed that one number cannot describe a classifier on imbalanced data. The remedy is not a better single number — it is to stop summarising prematurely. Every metric in this section is built from the same four counts, and the counts come first because which metric you should care about is a question about *which of the four mistakes hurts*, and no formula answers that for you.

6.1 Four numbers

Stop collapsing the result into one number and count all four outcomes.

	Predicted healthy	Predicted failing
Actually healthy	TN — true negative	FP — false positive
Actually failed	FN — false negative	TP — true positive

The names mean nothing until attached to consequences, so attach them:

- **False positive**: we replace a healthy drive. Cost: a technician's hour and the price of a drive.
- **False negative**: a drive fails in service unwarned. Cost: an emergency callout, a degraded array, possibly data.

For our model at threshold 0.5, as notebook 2 counts them:

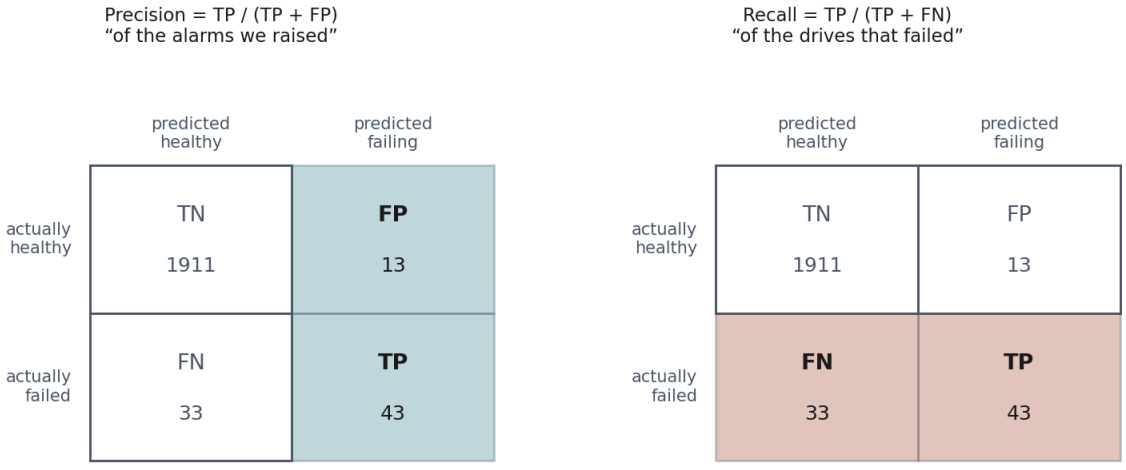
	pred. healthy	pred. failing
healthy	1911	13
failed	33	43

Read the bottom-left cell first when misses are expensive: **33 drives failed and were called healthy.** The top-right cell is what the 43 caught cost: 13 healthy drives flagged.

A predictable mistake: "33 misses, so the model needs improving". Perhaps, but that is not the first lever. The probabilities stay the same and 0.5 is a habit: a lower threshold catches more failures with the same model, at the price of more false alarms (section 7). The threshold trades one error for the other; only a better model, one that ranks the drives better, reduces both.

6.2 Precision and recall

Both divide the true positives. They differ in the denominator, and the denominator is the whole meaning.



*The difference is entirely the denominator. Precision divides by the shaded **column**, everything we flagged; recall divides by the shaded **row**, everything that actually failed.*

$$\text{precision} = \frac{TP}{TP + FP} \qquad \text{recall} = \frac{TP}{TP + FN}$$

Precision divides by everything we flagged. *When this model raises an alarm, how often is it right?* It is the technician's question: how much of my time are you wasting?

Recall divides by everything that was truly positive. *Of the drives that were going to fail, how many did we find?* It is the operations manager's question: how exposed am I still?

A hook that survives exam pressure: **precision is about the alarms, recall is about the failures.** Look at the denominator and ask which population you are dividing by.

For our model: precision $= 43/(43+13) = 43/56 = 0.768$, recall $= 43/(43+33) = 43/76 = 0.566$. Of the 56 drives flagged, 43 really failed; of the 76 that failed, we found 43.

Two further quantities appear often enough to name:

$$\text{specificity} = \frac{TN}{TN+FP} = \frac{1911}{1911+13} = 0.993 \qquad \text{false positive rate} = 1-\text{specificity} = \frac{13}{1924} = 0.007$$

6.3 F1, and why the harmonic mean

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

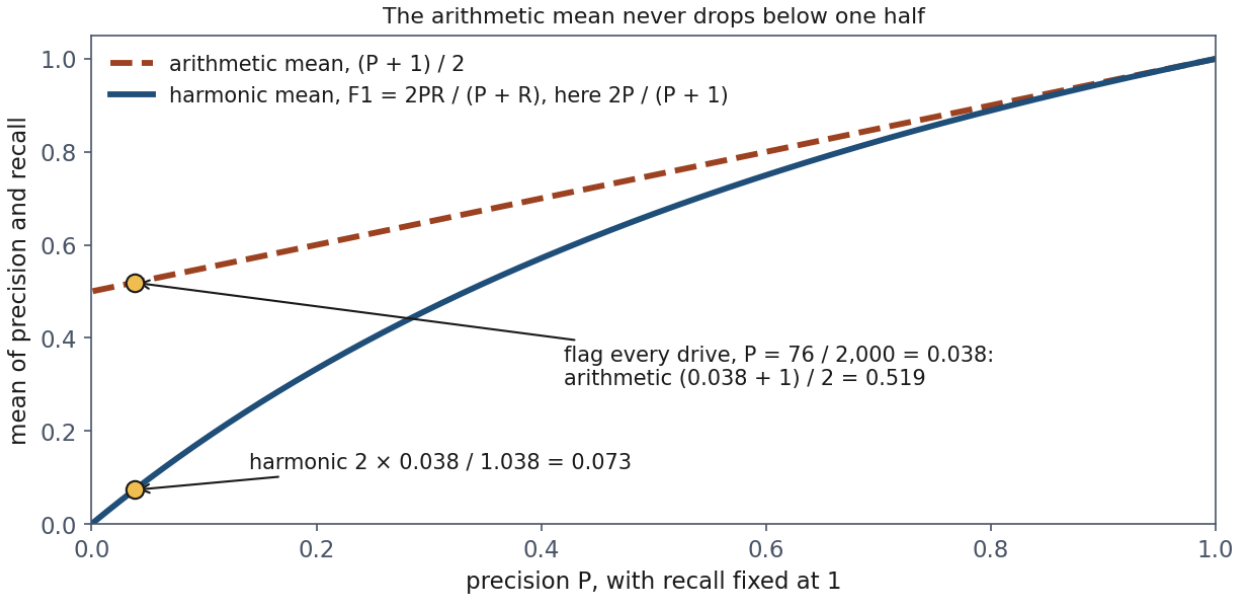
That expression is the **harmonic mean** of the two, which is easier to recognise in the form it is defined by — invert both, average, invert back:

$$\frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

multiplying numerator and denominator by the product of the two. Inverting is what makes it behave differently from an ordinary average: a small number has a large reciprocal, so the *worse* of the two dominates the sum in the denominator.

Why not the ordinary average? Consider the model that flags every drive. Recall is a perfect $76/76 = 1$; precision is $76/2000 = 0.038$, because every one of the 2,000 drives is flagged and 76 of them fail.

Mean	Value
Arithmetic	$(0.038+1)/2 = 0.519$
Harmonic (F_1)	$2 \times 0.038 \times 1/(0.038+1) = 0.073$



Both means for a model with perfect recall, as its precision varies. Look at the left edge, where the flag-everything model sits: the arithmetic mean never drops below one half, the harmonic mean drops with the precision.

The arithmetic mean gives a useless model a respectable-looking score, because a perfect result on one axis pays for a catastrophe on the other. The harmonic mean is dominated by the smaller number: **a model is only as good as its weaker side.**

A second reason, in counts. Substituting the definitions,

$$F_1 = \frac{2TP}{2TP + FP + FN} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

Precision and recall share the numerator TP and differ in the denominator, and for two such ratios the harmonic mean is the one that is still a ratio of drives: TP over the average of the two denominators, $TP + FP$ and $TP + FN$. The arithmetic mean of two ratios counts nothing. For our model, $2 \times 43/(2 \times 43 + 13 + 33) = 86/132 = 0.652$, the F_1 of section 6.4.

A predictable mistake: "at $P = 0$ the arithmetic mean is 0.5 and F_1 is 0.073". The 0.073 belongs to the flag-everything model, $P = 0.038$; at $P = 0$ exactly, F_1 is 0.

The generalisation, when the two are not equally important, is the same reciprocal average with the two reciprocals given unequal weights — 1 and β^2 out of $1 + \beta^2$:

$$F_\beta = \frac{1 + \beta^2}{\frac{1}{\text{precision}} + \frac{\beta^2}{\text{recall}}} = (1 + \beta^2)\,\frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

Setting $\beta = 1$ gives equal weights and recovers F_1, which is the check worth doing on any weighted formula. $\beta > 1$ puts more weight on recall's reciprocal, so a poor recall hurts more — F_2 is a common choice when misses matter more, as here — but β expresses a preference, not the costs: nothing turns 2600/140 into a β, and section 7.2 acts on the costs directly. $\beta < 1$ favours precision.

The square is there so that β reads as a ratio of importance rather than of reciprocals: at $\beta = 2$, recall is worth $\beta^2 = 4$ times precision in the sum, which is the convention the name carries.

A predictable mistake: "F_2 multiplies recall by 2". It is recall's reciprocal that gets the weight $\beta^2 = 4$, against 1 for precision's: $F_2 = 5PR/(4P + R)$. For our model, $5 \times 0.768 \times 0.566/(4 \times 0.768 + 0.566) = 0.597$ (from $P = 43/56$ and $R = 43/76$), below $F_1 = 0.652$ because recall is our weak side.

6.4 Averaging across classes

Scikit-learn's `classification_report` — the summary table notebook 2 prints, one row per class followed by three summary rows — is where the numbers below come from. Choosing between those three rows matters more than it looks.

Average	What it does	Our model
macro	mean over classes, each equal	$(0.988 + 0.652)/2 = 0.820$
weighted	mean over classes, weighted by support (the number of examples in each class)	$(1924 \times 0.988 + 76 \times 0.652)/2000 = 0.975$
micro / accuracy	pool all predictions	$(43 + 1911)/2000 = 0.977$

The per-class F_1 values come from the report's rows: 0.988 for the healthy class, 0.652 for the failed one ($2 \times 0.768 \times 0.566/(0.768 + 0.566)$). The weighted average is dominated by the healthy class, 1,924 of the 2,000 test drives, so it reads close to accuracy and inherits its blindness. **On an imbalanced problem, quote the minority class directly, and the macro average when the classes should count equally.** Reporting the weighted average is not incorrect, but it is the number that makes a mediocre model look finished.

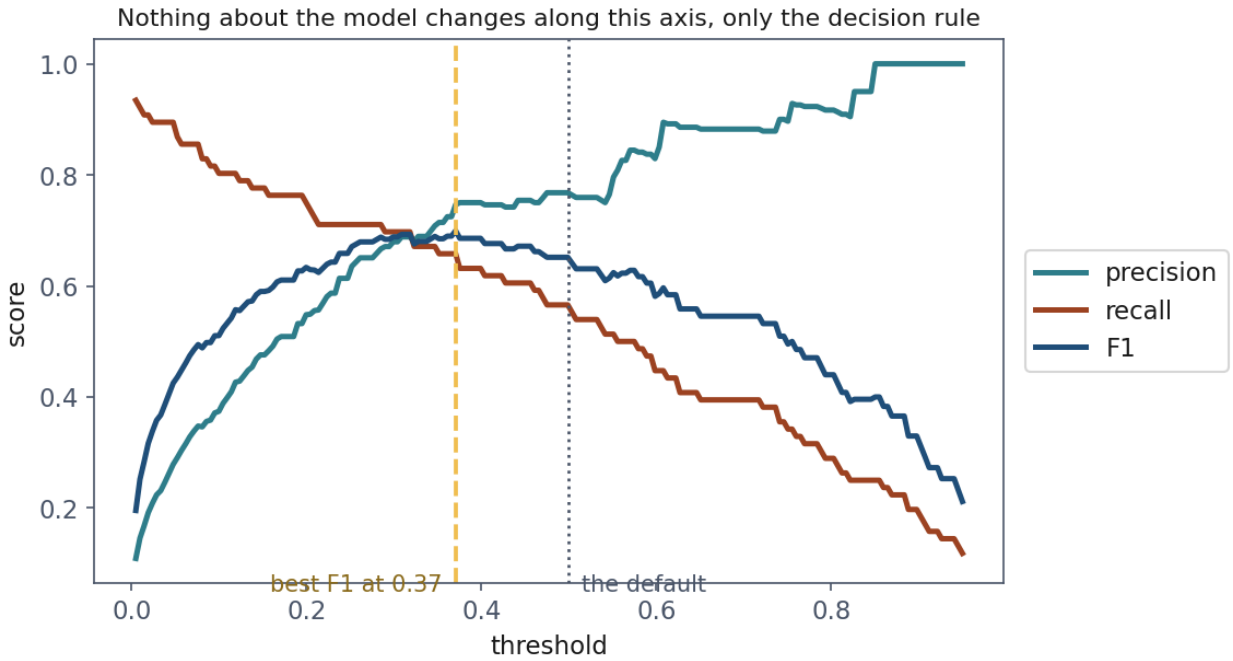
A predictable mistake: "macro average is a new metric". Macro and weighted are two ways of averaging over classes, and `classification_report` applies both to every column — precision, recall and F_1. And why is an arithmetic mean fine here, after section 6.3 rejected it? There it combined two different quantities of one class; here it is the same metric on two classes, and the point is to give the rare class an equal voice.

---

7. The threshold is a decision

7.1 Nothing about the model changes along this axis

Every number in section 6 depended on a rule we never examined: flag the drive when $\hat{y} \geq 0.5$.



The fitted model is identical at every point on this axis: same coefficients, same probabilities, same drives. Only the line between "leave it" and "replace it" moves.

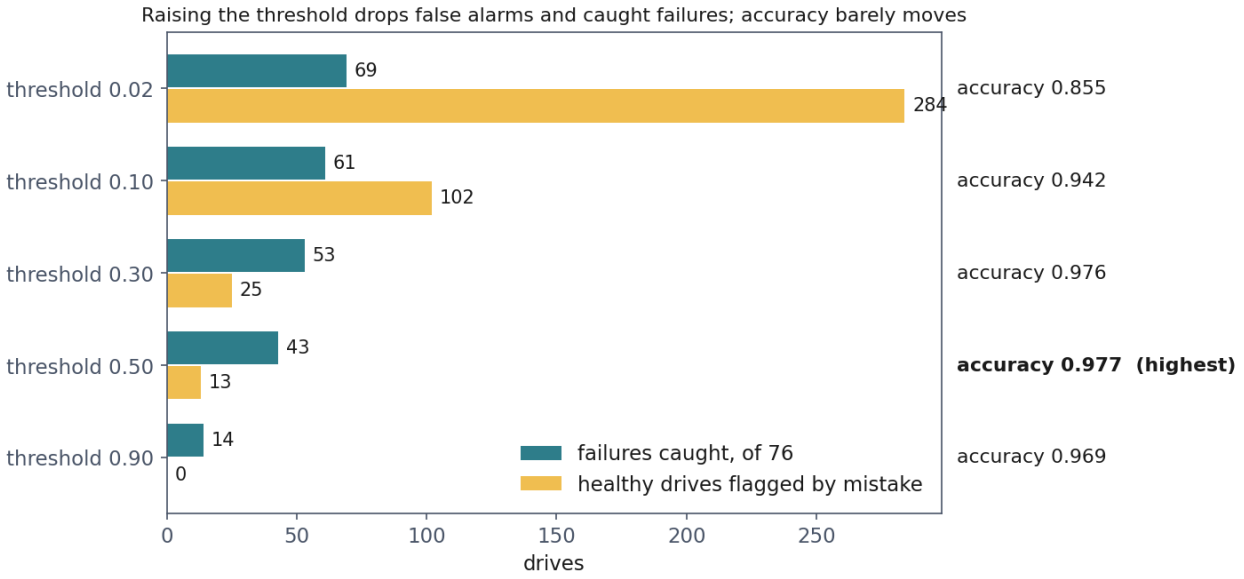
That 0.5 is `predict()`'s default and nothing more. It is the right choice only when the two errors cost the same and the probabilities are calibrated (among drives given 0.3, about 30% really fail): the threshold derived from the costs below is 0.5 exactly when the costs are equal, whatever the class balance. Here a missed failure costs about nineteen times a needless replacement ($2600/140 = 18.6$, section 7.2). The model's actual output is a probability; **turning it into a decision requires a threshold, and the threshold is a business question.**

Threshold	Flagged, TP + FP	TP	FP	FN	Precision, $\frac{TP}{TP+FP}$	Recall, $\frac{TP}{TP+FN}$	F_1, $\frac{2PR}{P+R}$	Accuracy, $\frac{TP+TN}{2000}$
0.02	353	69	284	7	0.195	0.908	0.322	0.855
0.05	235	67	168	9	0.285	0.882	0.431	0.911
0.10	163	61	102	15	0.374	0.803	0.510	0.942
0.20	104	57	47	19	0.548	0.750	0.633	0.967
0.30	78	53	25	23	0.679	0.697	**0.688**	0.976
0.50	56	43	13	33	0.768	0.566	0.652	**0.977**
0.70	34	30	4	46	0.882	0.395	0.545	0.975
0.90	14	14	0	62	1.000	0.184	0.311	0.969

The same fitted model produced every row. Same coefficients, same probabilities, same drives. Only the line between "leave it" and "replace it" moved.

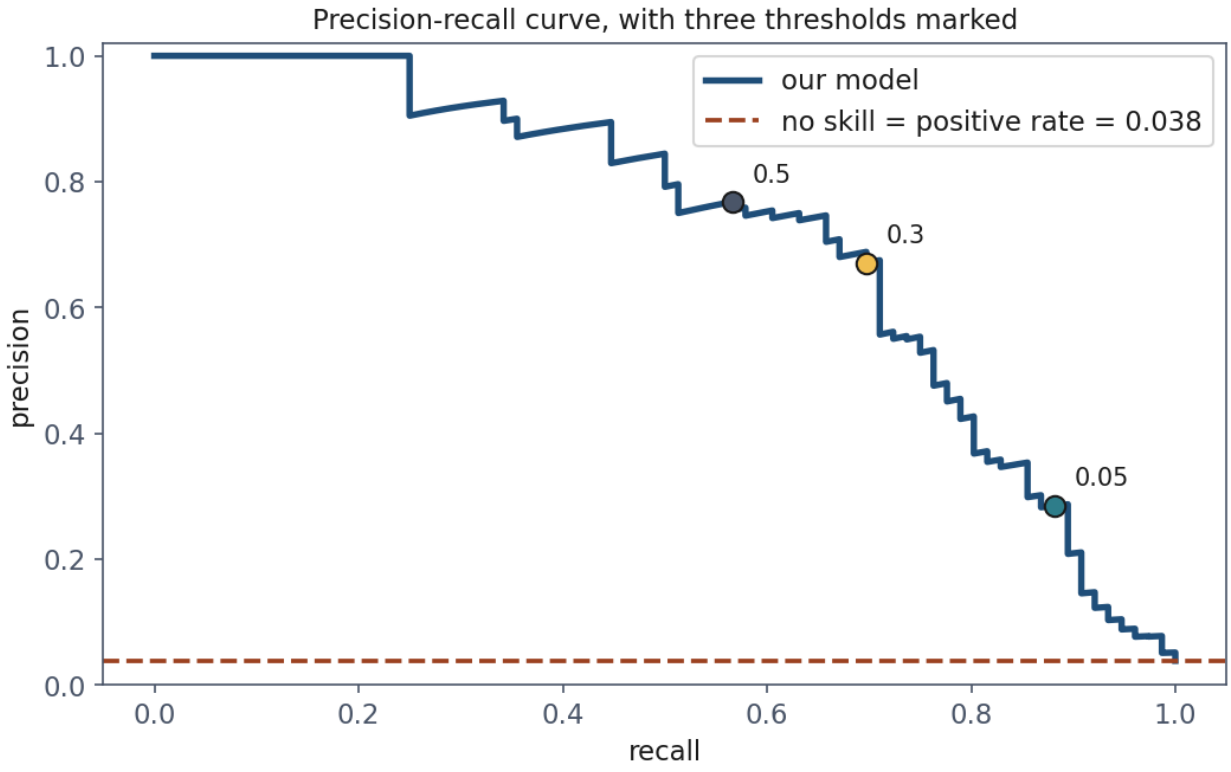
The counts come from notebook 2's sweep; TN, not shown, is the 1,924 healthy test drives minus FP. Note the accuracy column once more. From 0.20 upwards it varies by $0.977 - 0.967 = 0.010$, one percentage point, while recall falls from 0.750 to 0.184, and its maximum sits at the default

threshold, which misses 33 failures. **Tuning on accuracy quietly recommends catching fewer failures**, because most of the alarms a higher threshold drops are false ones.



Five rows of the table as bars. Look at the two bars shrinking together as the threshold rises — teal, the failures caught; gold, the false alarms — while the accuracy on the right barely moves and peaks at 0.5.

So when a paper or a colleague reports 77% precision (our 0.768 at 0.5), the useful question is: *at what threshold, and why that one?*



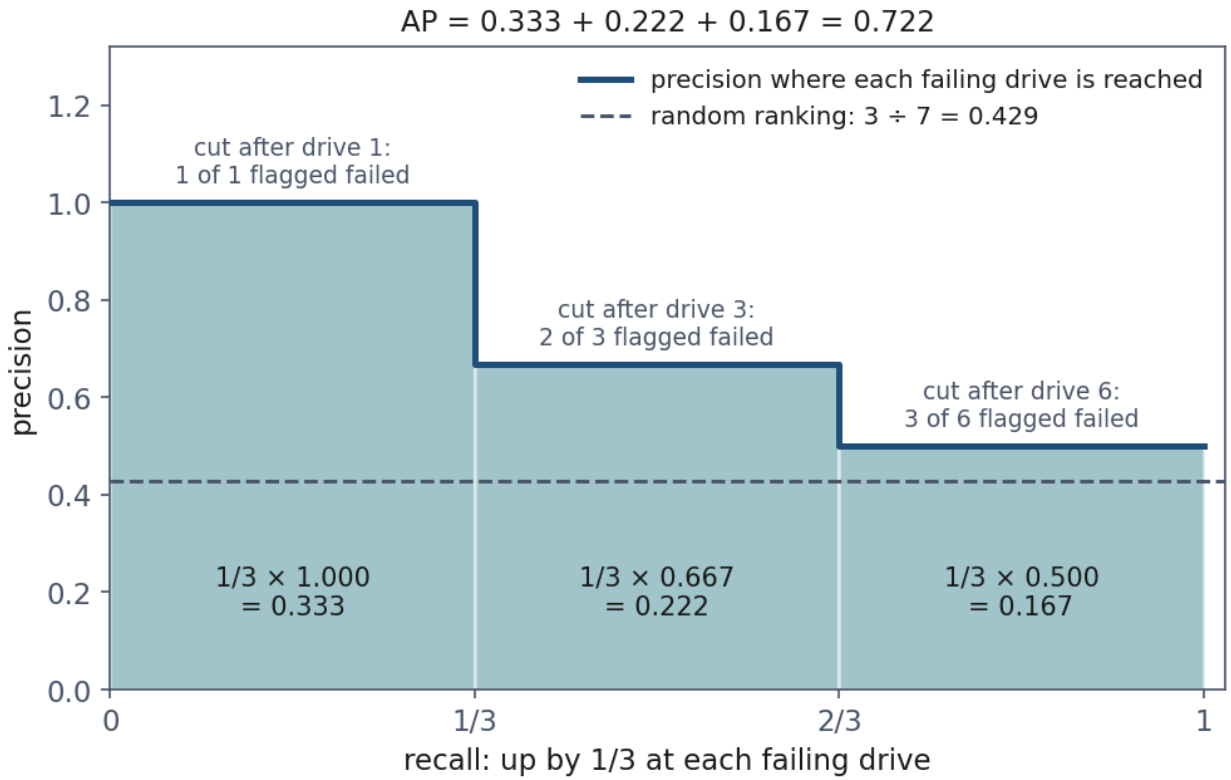
The table above, drawn as a curve, with three of its rows marked. Each point is one threshold; moving along the curve is the trade the table tabulates. The dashed line is what "no skill" looks like on this problem — the positive rate, 76 / 2,000 = 0.038, not 0.5 — and Section 8 returns to why that baseline matters more than the curve's shape.

Where the flat baseline comes from. A model with no skill flags drives at random, which is like drawing drives blind from an urn: a random selection keeps the proportions of what it is drawn from. Flag k of the 2,000 at random and, on average, the share of failures among them is the test set's share, $76/2000 = 0.038$, whatever k is: the expected number of failures is $k \times 0.038$, so precision is $(k \times 0.038)/k = 0.038$, and k cancels. Note that 0.038 is a share of the drives flagged, not a number of drives. Flag 500: about $500 \times 0.038 = 19$ of them fail, so precision is $19/500 = 0.038$ and recall $19/76 = 0.25$. Flag 1,000: about 38 fail, precision $38/1000 = 0.038$ again, recall $38/76 = 0.50$. Flagging more moves recall along the axis and leaves precision where it was: a horizontal line at the positive rate. It sits at 0.5 only on a balanced problem.

The area under the curve: average precision. Walk down the thresholds from the top, $k = 1, 2, ...$, with precision P_k and recall R_k at each. Every time recall goes up, take the precision at that point, weighted by the step:

$$\text{AP} = \sum_k (R_k - R_{k-1})\, P_k$$

The **average precision (AP)** is a weighted average of precisions, the weights being the recall each threshold adds. Recall goes up only when the cut passes a failing drive, and always by the same step, $1 \div 76$, so the weights are equal and AP is simply **the average of the precisions measured at each failing drive's place in the ranking**: each time the model reaches a failure, how clean is the queue of alarms at that moment? Seven drives make it visible:

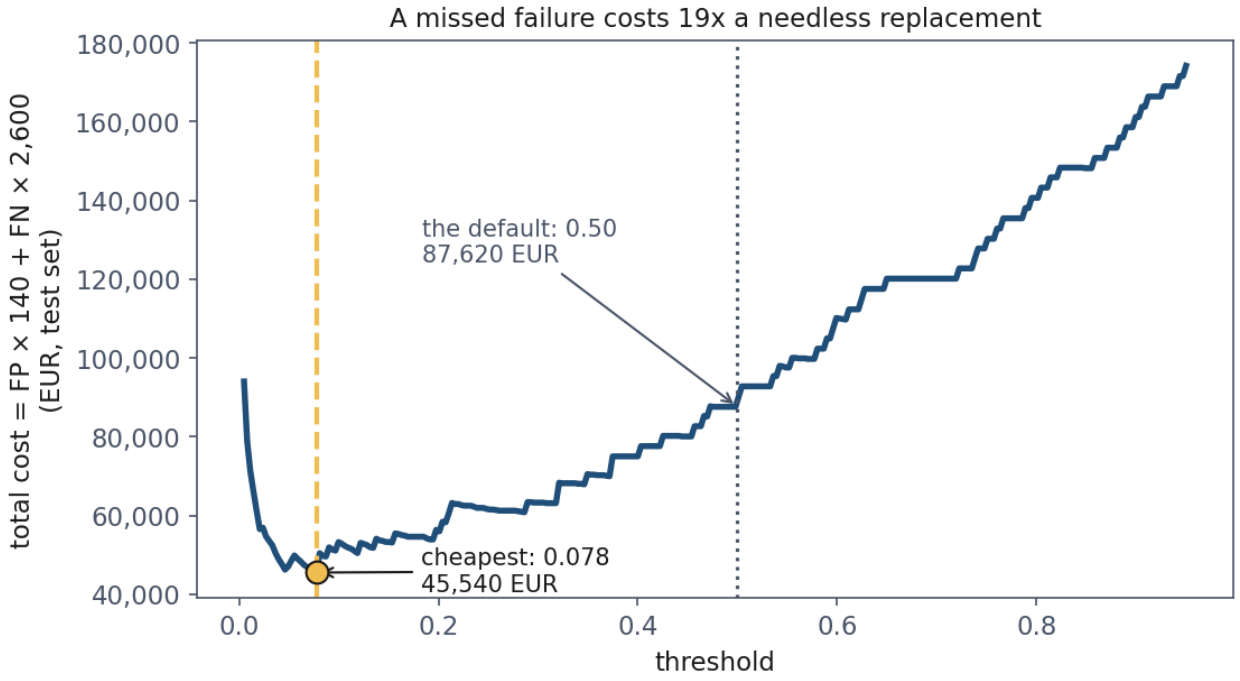


Seven drives ranked riskiest first: failed, healthy, failed, healthy, healthy, failed, healthy. Look at the three rectangles, one per failing drive: each is 1/3 wide, the recall it adds, and as tall as the precision where that drive is reached, 1 of 1, 2 of 3, 3 of 6. Their areas add up to the AP, 0.722, against the random ranking's 3 ÷ 7 = 0.429.

For our model it is **0.717** (`average_precision_score`, notebook 2). For the no-skill model of the previous paragraph the precision is the positive rate $\pi = 0.038$ at every recall, and a weighted average of a constant is that constant: its AP is π. So AP is read against the positive rate, never against 0.5.

7.2 Choosing the threshold from costs

A principled answer needs a criterion that no metric supplies. The one used here is **what the errors cost**; other constraints, such as how many alarms the technicians can follow up, may justify a different threshold (the homework has one). Suppose



Total cost against threshold, with a miss priced at nineteen false alarms (2,600 / 140 = 18.6). The minimum is far below 0.5, and the curve is nearly flat around it — so the exact value does not have to be right, which is fortunate, because cost estimates never are.

$$C_{FP} = 140 \text{ EUR} \qquad C_{FN} = 2600 \text{ EUR}$$

A missed failure costs about nineteen needless replacements, $2600/140 = 18.6$. Now the threshold follows from arithmetic rather than habit.

The derivation. For a drive with estimated failure probability p, flagging it has expected cost $(1-p)\,C_{FP}$ — we pay for a replacement only if the drive was in fact healthy. Leaving it alone has expected cost $p\,C_{FN}$. Flag when flagging is cheaper:

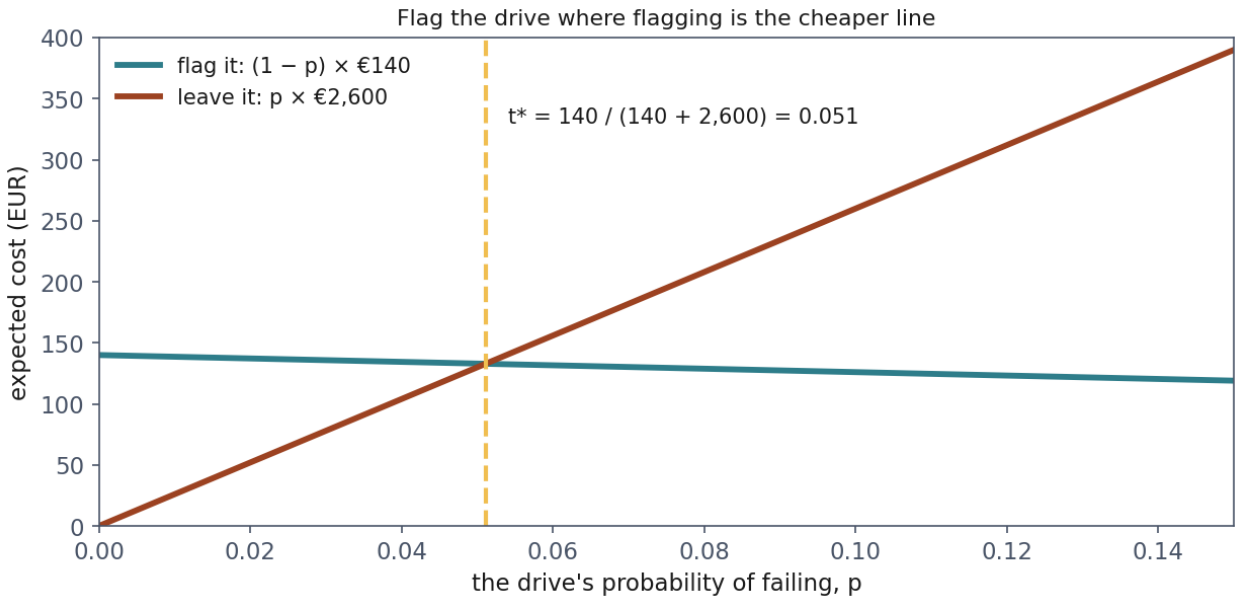
$$(1-p)\,C_{FP} < p\,C_{FN}$$

Multiply out the left-hand side and collect the two terms in p on one side: $C_{FP} < p\,C_{FN} + p\,C_{FP} = p\,(C_{FN} + C_{FP})$, so

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}} = t^*$$

The probability does not simplify away: it is the unknown being solved for, and t^* is a value of p — the one at which the two expected costs are equal. The rule reads: flag every drive whose p is above t^*.

$$t^* = \frac{140}{140 + 2600} = 0.051$$



The two expected costs of section 7.2 against the drive's probability p. Look at where the lines cross: below t^* leaving the drive alone (rust) is cheaper, above it flagging (teal) is.

Read the formula before the number. The optimal threshold does not depend on the model, the dataset, or the class balance — only on the *ratio* of the two costs. If misses and false alarms cost the same, $t^* = 0.5$ and the default is correct. As misses grow more expensive, t^* falls towards zero.

A predictable mistake: "on an imbalanced problem the threshold belongs below 0.5". Every threshold this lesson moves goes down, so the pattern is easy to generalise. But the class balance is not in the formula: with calibrated probabilities and equal costs, t^* is 0.5 however rare the failures are. The threshold falls here because a miss costs 18.6 times a false alarm.

And the two correct decisions? The derivation gave them no cost, and a student will reasonably ask whether replacing a drive that really was failing costs nothing. It costs a technician's hour, but a failing drive has to be replaced anyway: the model did not cause that cost. Each figure above is what an error adds over the right decision, so correct decisions add nothing. With all four outcomes priced, the same argument gives

$$t^* = \frac{C_{FP} - C_{TN}}{(C_{FP} - C_{TN}) + (C_{FN} - C_{TP})}$$

which only differences enter, and which is $140/(140 + 2600) = 0.051$ again when $C_{TP} = C_{TN} = 0$.

The condition the derivation used. It treated p as the true probability that the drive fails. That holds only if the model is **calibrated**: among the drives it scores 0.05, about 5% really fail. Section 8.2 divided every probability by ten without changing the AUC; that model would still be told $t^* = 0.051$, and would flag far too few drives. The formula is only as good as the probabilities fed to it.

On the test set, sweeping the threshold empirically puts the cheapest value at **0.078**, close to the theoretical 0.051, and the cost curve is nearly flat between them:

Policy	Cost on 2,000 drives, $FP \times 140 + FN \times 2600$	FP	FN
Do nothing	197,600 EUR	0	76
Threshold 0.50	87,620 EUR	13	33
Threshold 0.078	**45,540 EUR**	121	11

Choosing the threshold on purpose halved the cost of the same model: 45540/87620 = 0.52. No retraining, no new features, no better algorithm — one number, chosen from the economics instead of from the library default.

Two honest caveats.

The flatness is a feature. Because the curve is nearly flat near its minimum, the exact threshold does not have to be right. That is fortunate, because cost estimates never are.

What separates 0.051 from 0.078, counted in notebook 3. At exactly $t^* = 140/2740$ the test set gives FP 165 and FN 10, a cost of $165 \times 140 + 10 \times 2600 = 49{,}100$ EUR. Between t^* and 0.078 lie 45 test drives, and 1 of them failed. Flagging them costs $44 \times 140 = 6{,}160$ EUR; leaving them cost that one failure, 2,600 EUR, so on this test set leaving them was cheaper. But their probabilities add up to 2.76 expected failures: for a calibrated model, leaving them costs about $2.76 \times 2600 = 7{,}176$ EUR in expectation, against $(45 - 2.76) \times 140 = 5{,}914$ EUR for flagging, and the rule is right. The gap is 45 drives with one failure where the model expected almost three: sampling noise on 76 failures, plus calibration that is good but not perfect.

This threshold was chosen on the test set, which is exactly what lesson 1 told you not to do. Properly, it is chosen on a validation set and only then measured on test data. Lesson 5 builds that machinery; until then this is a demonstration of a method, not a number you could report. The two costs above show what looking is worth: t^*, fixed by the costs before anyone looked, costs 49,100 EUR on the test set; 0.078, the best of 300 thresholds tried on it, 45,540 EUR. Part of the 3,560 EUR (8%) is luck the test set handed us, and the minimum of many tries on the same data is biased downwards. A test score is an unbiased estimate only if the test set played no part in any choice and is used once, at the end.

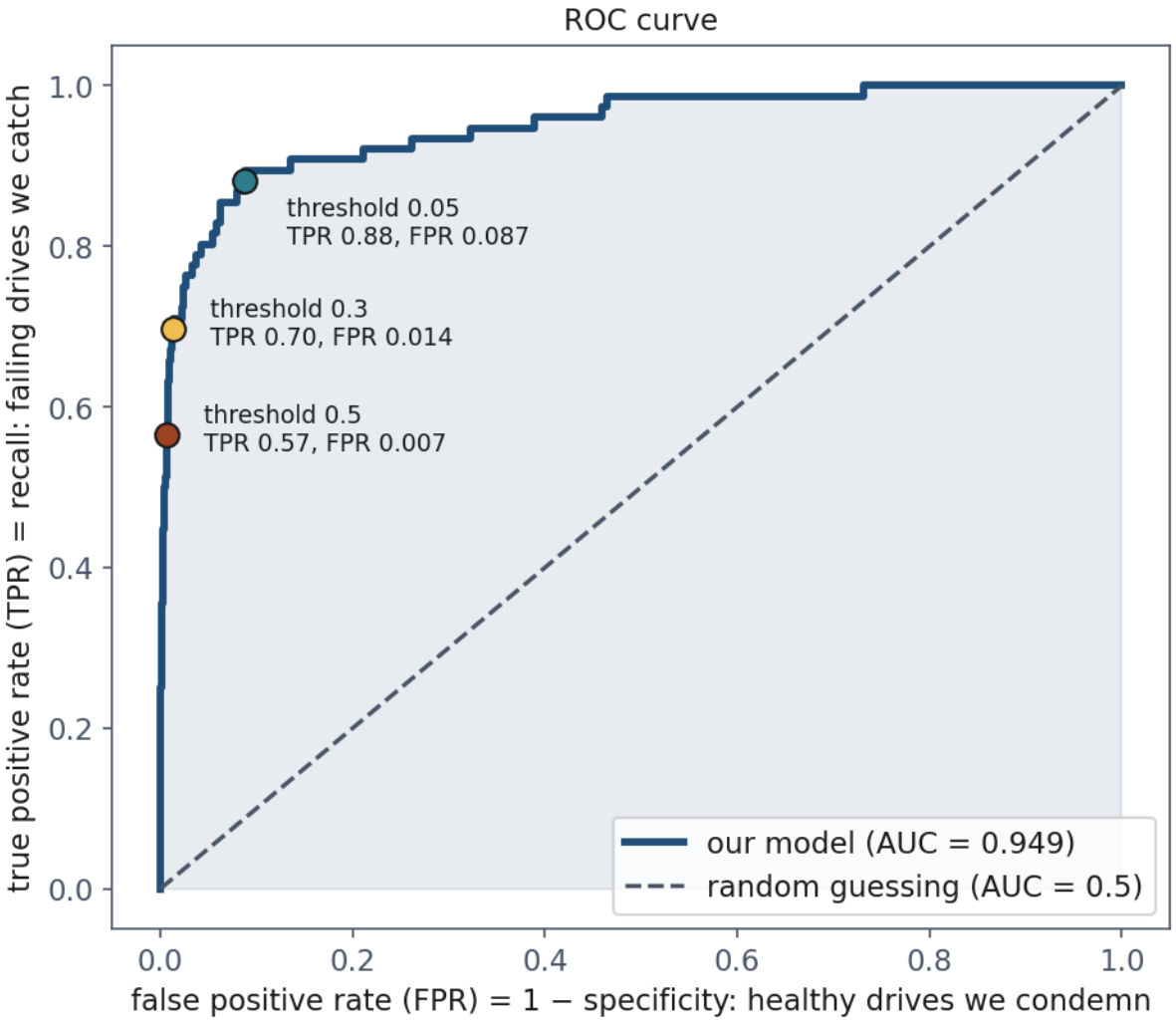
So which threshold is deployed? 0.051, or one chosen on validation data. A predictable mistake is to pick 0.078 because it costs less. It does, on the 2,000 drives it was chosen on; over new drives, the expected costs above say the rule of t^* is the right one, and 0.078's advantage was this test set's luck.

---

8. ROC and AUC

8.1 The curve

Start from the **ranking**: the 2,000 test drives sorted by the probability the model gave them, riskiest first. A threshold is a cut in that list — every drive above the cut is flagged — and lowering the threshold moves the cut down. So rather than pick one cut, look at all of them: sweep every threshold and plot two rates against each other. (The precision-recall curve of section 7.1 already swept every threshold; the reading here is different, as the end of this section says.)



The three marked points are three thresholds on one curve. The diagonal is guessing; the top-left corner is perfection; the area between says how far towards the corner this model got.

$$\text{TPR} = \frac{TP}{TP + FN} = \text{recall} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

The **true positive rate (TPR)** is the fraction of failing drives caught; the **false positive rate (FPR)** is the fraction of healthy drives wrongly condemned. The plot is the **receiver operating characteristic (ROC)** curve — the name is wartime radar, where operators tuned receivers to spot aircraft without chasing flocks of birds.

The intuition. At threshold 1 both rates are 0: flag nothing, catch nothing, disturb nobody. At threshold 0 both are 1. The whole content of the curve is *the order in which they get there.* A good model raises TPR fast while FPR is still low, because its highest scores really are the failures. A useless model raises them together and traces the diagonal.

So the curve asks: **as we grow more willing to raise alarms, do we catch failures faster than we annoy technicians?**

The steps, counted. Walk down the ranking one drive at a time. A failing drive moves the curve

up by 1/76, a healthy one moves it right by 1/1,924, so a steep start means the top of the ranking is failures. The ideal curve goes straight up to $(0, 1)$ and then across. Every curve ends at $(1, 1)$, so a curve that is steep at the start is flat at the end, and the area under it (section 8.2) measures exactly that. One point checked: at threshold 0.5, TPR = 43/76 = 0.57 and FPR = 13/1,924 = 0.007.

Same sweep as the precision-recall curve, different reading. The precision-recall curve's ideal is the top-right corner, and precision divides by the alarms raised. The ROC curve's ideal is the top-left corner, and FPR divides by the 1,924 healthy drives, so its random baseline is the diagonal whatever the class balance, against 0.038 for precision-recall. That denominator is why ROC can hide imbalance (section 8.3).

8.2 What the area means

The **area under the curve (AUC)** summarises the whole curve in one number: 0.5 for guessing, 1.0 for perfection. Ours, from `roc_auc_score` in notebook 3, is **0.949**.

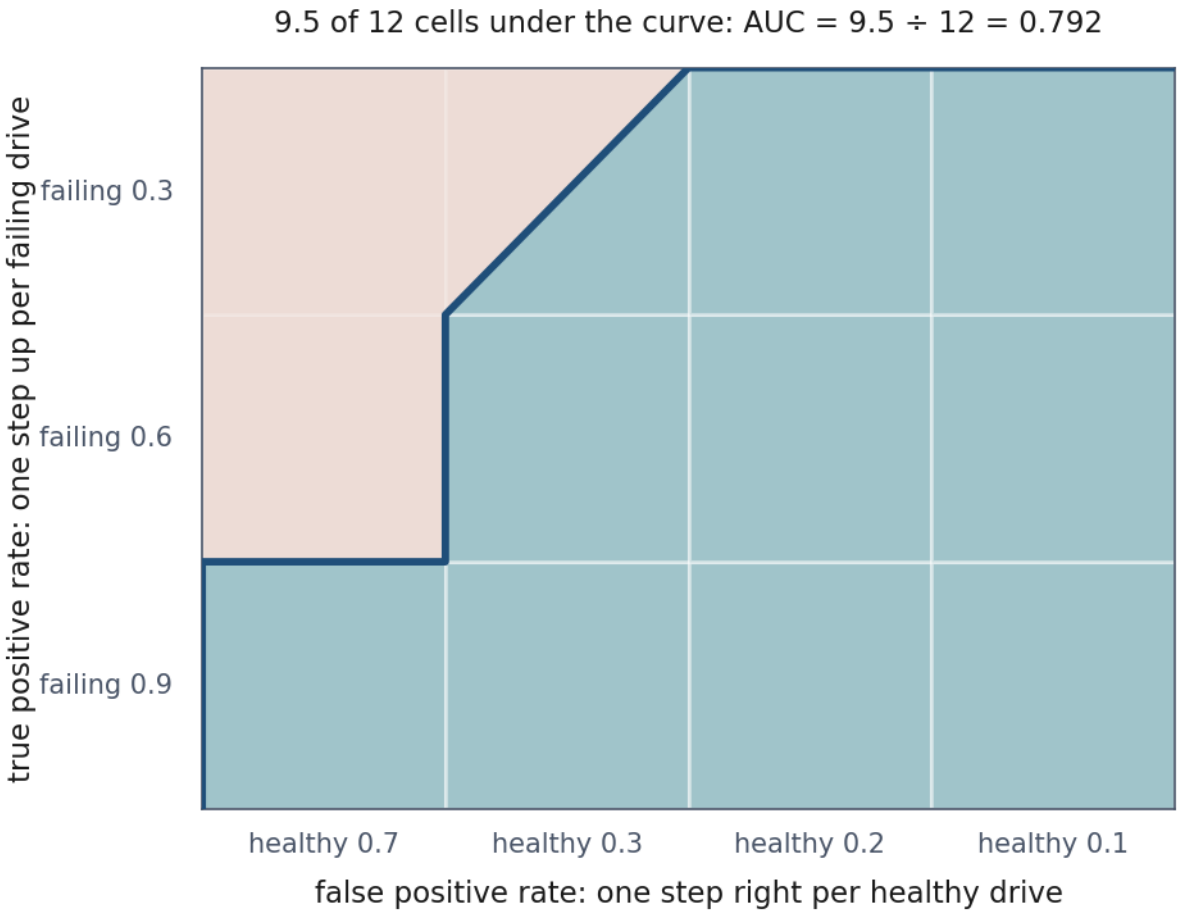
The area has an exact probabilistic meaning, and it is the thing worth remembering:

> **AUC is the probability that the model assigns a higher score to a randomly chosen positive than to a randomly chosen negative.**

That is, pick one drive that failed and one that did not, at random; AUC is the probability the failing one scored higher.

Why the area is that probability: the ROC curve is a staircase on a grid. Three steps.

1. **The grid.** Sort the drives by score, riskiest first, and walk down the list. Each failing drive moves the curve up by 1/76, each healthy drive moves it right by 1/1,924. The unit square is therefore cut into 76 rows and 1,924 columns: $76 \times 1{,}924 = 146{,}224$ cells, one for every (failing, healthy) pair, each of area $1 \div 146{,}224$.
2. **Which cells lie under the curve.** Take the column of one healthy drive. The curve steps right across that column at the height it has reached so far: (failing drives scored above it) $\div$ 76. So the cells under the curve in that column are exactly the pairs in which a failing drive beats this healthy one. A tie is a diagonal step, which cuts its cell in half: a tie counts half.
3. **Add up the columns.** The area under the curve is (pairs in the right order) $\div$ (all pairs): the probability that a random failing drive outscores a random healthy one. In statistics this count is the Mann-Whitney U statistic.



Seven drives: three failing, scored 0.9, 0.6 and 0.3, and four healthy, scored 0.7, 0.3, 0.2 and 0.1, so 3 × 4 = 12 cells. Look at the teal cells under the curve: in each the failing drive of its row scores above the healthy drive of its column. The cell of the tie at 0.3 is cut by the diagonal step and counts half. 9.5 of 12 cells: AUC = 0.792.

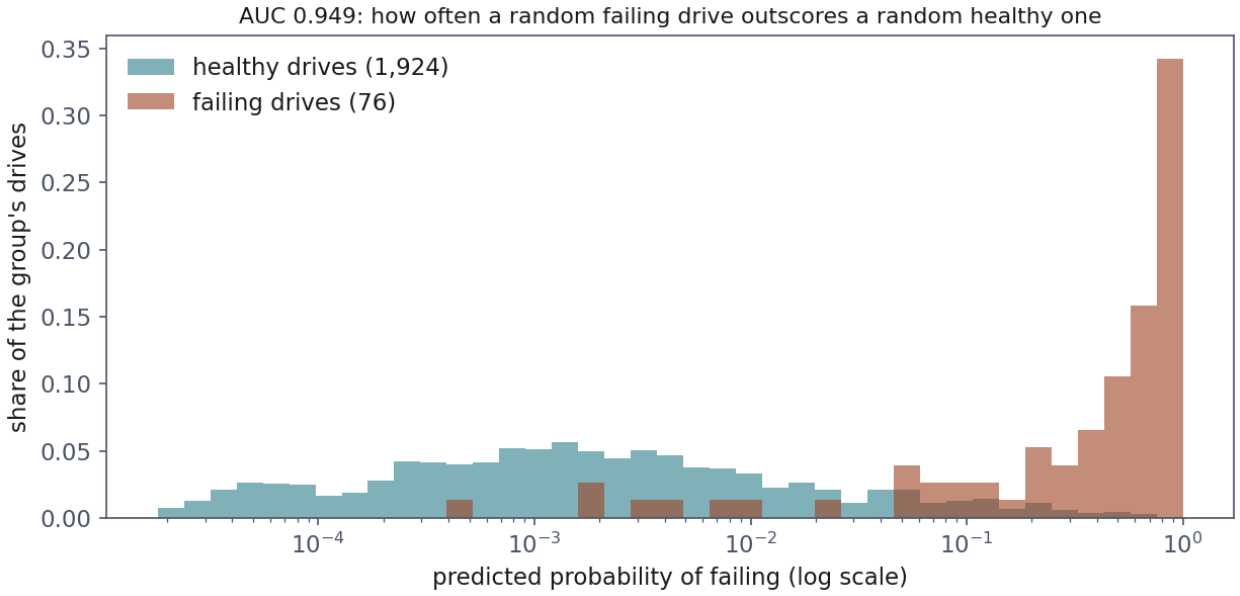
Column: healthy drive	Failing drives above it	Cells under the curve
0.7	0.9	1
0.3	0.9, 0.6, and the tie with 0.3	2 + ½ = 2.5
0.2	all three	3
0.1	all three	3

AUC = (1 + 2.5 + 3 + 3) ÷ 12 = 9.5 ÷ 12 = 0.792, the same number whether the pairs are counted or the area is measured.

Counted on our test set. There are $m_+ m_- = 76 \times 1{,}924 = 146{,}224$ (failing, healthy) pairs; a tie counts half. Notebook 3 counts 7,409 of them in the wrong order, the healthy drive scoring higher, so

$$\text{AUC} = 1 - \frac{7{,}409}{146{,}224} = 0.9493.$$

"Probability" and "share of pairs" are the same number. A failing drive costs one wrong pair for every healthy drive scored above it, so the errors are concentrated: the lowest-scoring failure ($p = 0.0004$) is outscored by 1,408 healthy drives, a fifth of all the wrong pairs on its own; the 5 worst failures make 61% of them, and 19 failures score above every healthy drive and cost nothing.



The scores the model gives the 76 failing drives (rust) and the 1,924 healthy ones (teal), on a logarithmic axis. Look at how far right the rust sits: pick one of each at random, and the AUC is the chance the rust one lies further right. The heights are shares of each group, not counts.

A predictable mistake: reading the heights as numbers of drives. A histogram usually counts, so the reading is natural. Here each colour adds up to 1, or the 76 rust drives would be invisible beside the 1,924 teal ones. Converted back in notebook 3, the tallest rust bar (0.342) is 26 drives and the tallest teal bar (0.056) is 108: the lower bar holds four times as many.

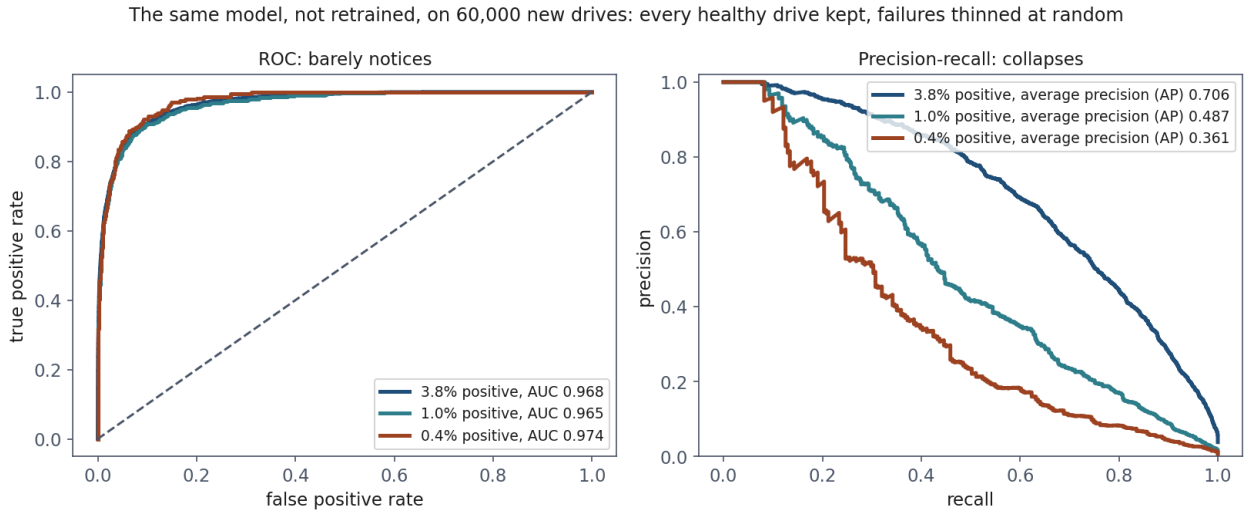
Notebook 3 also checks it by experiment rather than accepting the argument: drawing 200,000 random (failing, healthy) pairs, the failing drive scored higher in **0.9488** of them, against `roc_auc_score`'s **0.9493**. The draws estimate the share of the 146,224 pairs, so the gap is sampling noise.

A consequence that catches people out. AUC depends only on the *ranking*, so it is invariant to any strictly increasing transformation of the scores (a decreasing one reverses the order and turns the AUC into $1-\text{AUC}$). Divide every predicted probability by 10 and the AUC is unchanged, while the model becomes badly calibrated and every threshold-based decision changes. In notebook 3: at threshold 0.5 the fitted model flags $43 + 13 = 56$ drives; divided by 10 its largest score is 0.100 and it flags none, with the AUC still 0.949. **AUC says nothing about whether "0.7" means anything.** If you need the probabilities to be believable — because you are computing expected costs, as in section 7.2 — AUC is not the check you want.

The converse holds too, which shows the two properties are independent. Score every drive at 0.038, the fleet's failure rate: on average the score is right, so the model is calibrated in the large, and it ranks nothing, AUC 0.5.

8.3 Where ROC misleads

An AUC of 0.949 sounds like a finished project. Now make the positives rarer, which is the direction reality moves in fraud, disease and hardware failure.



One model, drawn twice, as the positives are thinned from 3.8% to 0.4%. On the left the three curves lie almost on top of each other. On the right they are in different worlds.

Keep the same model, generate a fleet of 60,000 drives, keep every one of its 57,575 healthy drives, and thin the failures until they are 0.4% instead of 3.8%: the three fleets hold 59,849, 58,157 and 57,806 drives. **The model is not retrained.** Notebook 3 computes every number in the table below.

Positive rate, failures / fleet	AUC	Average precision	Precision at 0.5, $\frac{TP}{TP+FP}$
3.8%	0.968	0.706	0.788
1.0%	0.965	0.487	0.462
0.4%	0.974	0.361	0.259

AUC does not move. It is a property of the ranking, and thinning positives at random does not change how the model ranks. It goes on reporting an excellent model — at the rarest setting, a marginally better one, which is sampling noise on 231 positives.

Precision falls to a third of what it was, $0.259/0.788 = 0.33$. At the same threshold, with the same model, $1 - 0.259 = 0.74$ — three alarms in four — are now false. That is the number the technician experiences.

Why the disagreement. Look at the counts at threshold 0.5. The false alarms are **304 in all three fleets**, because the healthy drives are the same 57,575 each time; FPR divides them by those healthy drives, $304/57575 = 0.0053$, and does not move. The true alarms fall with the failures, $1127 \to 261 \to 106$, while recall — a share of the failures — stays near 0.46–0.50. Both ROC axes are shares inside one class, so the ROC curve cannot move. Precision, $TP/(TP + 304)$, is the one number that mixes the classes, and it falls. In one formula, with π the positive rate,

$$\text{precision} = \frac{R\,\pi}{R\,\pi + \text{FPR}\,(1-\pi)}$$

R and FPR are the ROC's axes and stay put; only π moves. On the rarest fleet, $0.459 \times 0.004/(0.459 \times 0.004 + 0.0053 \times 0.996) = 0.26$.

A predictable mistake: "precision falls because the false positives grow". They do not — 304 each time. What grows is their *share* of the alarms, $304/(1127 + 304) = 0.21$ at 3.8% and $304/(106 + 304) = 0.74$ at 0.4%, which is 1− precision. The wording "the false alarms take over" invites the slip, because it sounds like "there are more of them".

This is the second predictable mistake of the lesson, and again the instinct is defensible: AUC is threshold-free, comparable across models and datasets, and bounded in a familiar range. Those are real virtues. It simply answers a question about ranking when the question you had was about the alarms someone will act on.

The rule: on an imbalanced problem, report both. The two measures answer two questions: AUC, whether the model puts failures above healthy drives; the precision-recall curve, how many of the alarms it raises are real. Report the precision-recall curve beside the AUC, not one in place of the other. Its baseline is the positive rate — 76 / 2,000 = 0.038 here, not 0.5 — and its area, the average precision of section 7.1, was 0.717 for our model.

---

9. Class weights and resampling

Moving the threshold changes the decision *after* training. **Class weights** change the training itself, by making each rare example count as several.

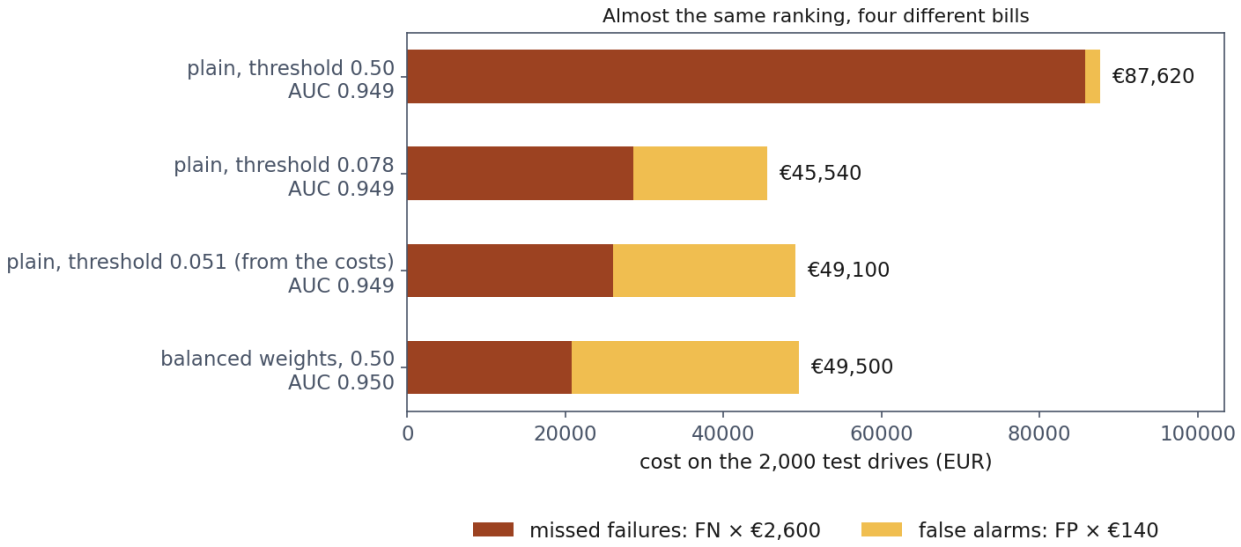
The weighted cost is

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m} c_{y^{(i)}}\left[\,y^{(i)}\log p^{(i)} + \left(1-y^{(i)}\right)\log\left(1-p^{(i)}\right)\right]$$

with `class_weight="balanced"` setting $c_k = m/(K\,m_k)$, where m_k is the number of examples of class k — inversely proportional to each class's frequency. Here that makes each failure count about 25 healthy drives: the ratio of the two weights is $m_0/m_1 = 5770/230 = 25$, the healthy drives in the training set over the failed ones. The name says what the formula does: weight times class size is m/K for every class, here $0.52 \times 5{,}770$ and 13.0×230, both about 3,000, so after weighting each class carries the same total weight, half each, however many examples it has.

Model	TP	FP	FN	Precision, $\frac{TP}{TP+FP}$	Recall, $\frac{TP}{TP+FN}$	AUC	Cost = FN × 2,600 + FP × 140
Plain, threshold 0.50	43	13	33	0.768	0.566	0.949	87,620

Model	TP	FP	FN	Precision, $\frac{TP}{TP+FP}$	Recall, $\frac{TP}{TP+FN}$	AUC	Cost = FN × 2,600 + FP × 140
Plain, threshold 0.078	65	121	11	0.349	0.855	0.949	45,540
Plain, threshold 0.051, from the costs	66	165	10	0.286	0.868	0.949	49,100
Balanced weights, 0.50	68	205	8	0.249	0.895	0.950	49,500



The same four models as bills. Look at how the bill splits: rust, the missed failures at €2,600 each; gold, the false alarms at €140 each. The AUC beside each name hardly changes, and the two bottom bars, the threshold from the costs and the balanced weights, cost practically the same: €49,100 and €49,500.

The fair comparison for the weighted model is the threshold fixed from the costs, 0.051, not 0.078, which was chosen by looking at the test set (section 7.2).

Look at the AUC column: 0.949 against 0.950. Reweighting taught the model essentially nothing new about disk failure. What it did was move where the model puts its 0.5 line — the same lever as the threshold, pulled at a different moment.

Why it behaves like a lower threshold. With weights $c_0 = 6000/(2 \times 5770) = 0.52$ and $c_1 = 6000/(2 \times 230) = 13.0$, the weighted cost is the plain one computed on a training set holding about 25 copies of every failure. For logistic regression the main effect is on the intercept, which notebook 3 finds rises from -6.09 to -2.95, by 3.14, close to $\log 25 = 3.22$: every log-odds goes

up by about $\log 25$, every odds is multiplied by about 25, and the weighted model's 0.5 (odds 1) lands where the plain model's odds were 1/25, a probability of $1/(1+25) = 0.038$. Checked on the test set: the plain model at 0.038 raises 208 false alarms and misses 8; the weighted model at 0.5 raises 205 and misses 8; the two agree on 1,987 of the 2,000 drives. In general, weights in the ratio k act approximately like a threshold of $1/(1+k)$: exactly only for a correctly specified model and a large sample, as nearly here, where the labels were generated by a logistic model in these very features. With a misspecified model, few data or a strong penalty, weights also move the slopes and the ranking.

A predictable mistake: reading "balanced" as "weighted by what the errors cost". Both are weights and both push towards the failures, so the reading is natural. But balanced weights come from the class frequencies — 25 here, whatever a miss costs. Weights from the costs are a separate choice, `class_weight={0: 140, 1: 2600}`, ratio 2600/140 = 18.6, which by the rule above acts like a threshold of $1/(1+18.6) = 0.051 = t^*$: notebook 3 finds 164 false alarms and 9 misses at 0.5 (46,360 EUR), against 165 and 10 for the plain model at t^*. Balanced weights land near the cost optimum here only because 25 happens to be close to 18.6.

That is worth knowing, because class weighting is often presented as a remedy for imbalance. It is not a remedy; it is another way of setting the same decision. The remedy, where one exists, is more positive examples.

Where it earns its place: when a downstream tool insists on `predict()` and gives you no access to a threshold, or when the model's probabilities are not usable. Then weighting is how you move the operating point.

The same caution applies to **resampling** — oversampling the minority class, or synthesising new minority examples with SMOTE (synthetic minority oversampling technique). It changes the effective threshold and can help an optimiser see a rare class at all, but it does not create information that was not in the data. And it must happen **inside the cross-validation fold**, never before the split, for exactly the reasons lesson 2 gave about imputation.

---

10. More than two classes (optional)

This section goes with the optional slides 58 to 60. Exercise 4 is a two-class problem and does not need it; lesson 9 introduces the softmax again.

Real fleets are not healthy-or-dead. A drive can be **degraded**: readable, but reporting errors and worth replacing at the next maintenance window rather than at three in the morning.

10.1 Softmax

For K classes the model produces one score per class and normalises:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad z_k = w_k^\top x + b_k$$

This is the **softmax**, and for $K = 2$ it reduces to the sigmoid — subtract z_0 from both scores and one of them becomes 0, leaving $\sigma(z_1 - z_0)$. The cost generalises to **categorical cross-entropy**:

$$J = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} \mathbb{1}\left[y^{(i)} = k\right] \log P\big(y = k \mid x^{(i)}\big)$$

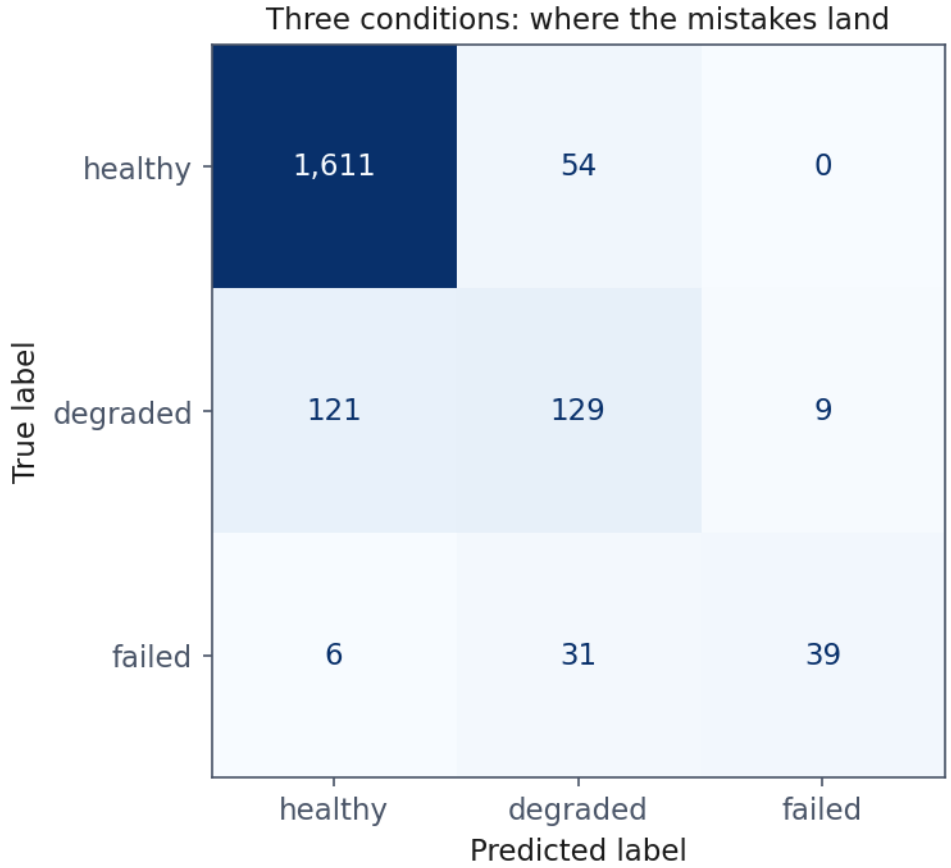
which is the same idea as before: the negative log of the probability the model assigned to what actually happened.

One drive, worked. Notebook 3 finds a degraded drive with scores $(0.615, 0.603, -1.218)$ for healthy, degraded and failed. Exponentiate: $e^{0.615} = 1.85$, $e^{0.603} = 1.83$, $e^{-1.218} = 0.30$; their sum is 3.97; divide: $(0.466, 0.460, 0.074)$, three positive numbers adding up to 1. The model calls the drive healthy, by 0.006. In scikit-learn, `decision_function` returns the scores and `predict_proba` their softmax; notebook 3 recomputes the softmax by hand and matches `predict_proba` to 3×10^{-16}.

The decision is the largest probability, so there is no 0.5 threshold with three classes: 33 test drives are assigned a class whose probability is below 0.5.

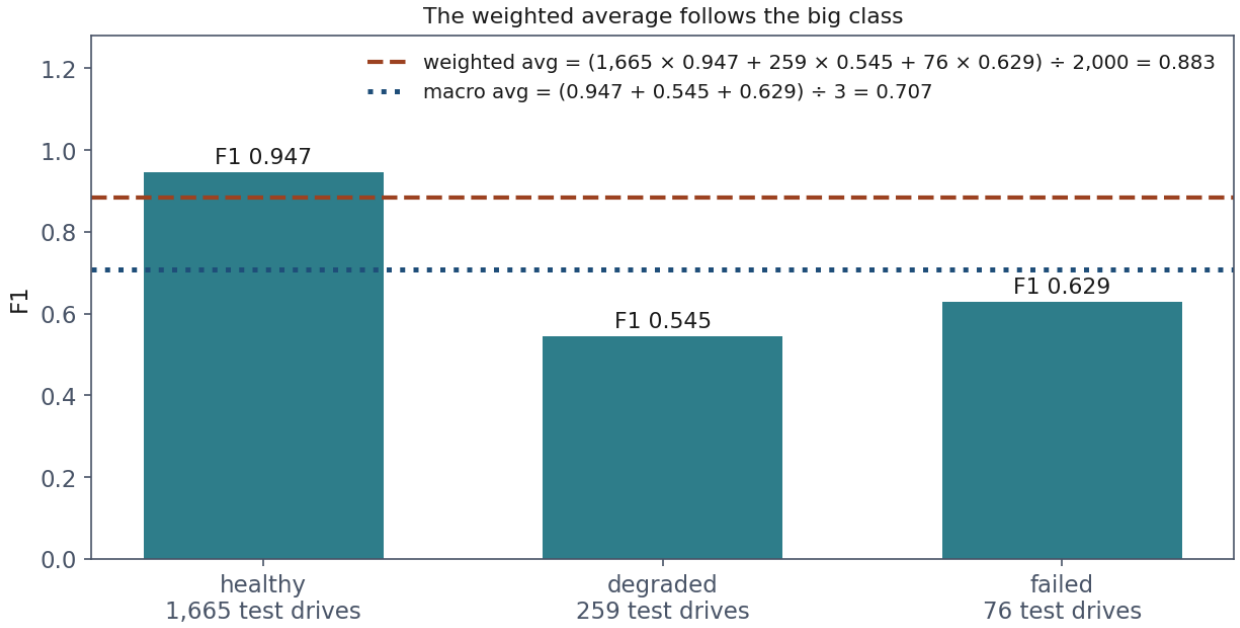
10.2 Metrics with three classes

The confusion matrix becomes $K \times K$, and precision and recall are computed **one class at a time**: for each class in turn, that class is positive and the rest are negative. On our three conditions, from notebook 3:



Read the off-diagonal cells rather than the accuracy. Degraded is the hard class: it sits between two neighbours that both resemble it, and the model confuses it in both directions.

Class	Precision	Recall	F_1	Support
healthy	0.927	0.968	0.947	1665
degraded	0.603	0.498	0.545	259
failed	0.812	0.513	0.629	76
macro avg	0.781	0.660	0.707	
weighted avg	0.881	0.889	0.883	



One bar per class. Look at where the two lines sit: the weighted average (dashed) close to the healthy bar, because healthy drives are 1,665 of the 2,000; the macro average (dotted) in the middle, because each class counts once.

The weights of the weighted average are the **support**: the number of test drives whose *true* class is that one, not the number predicted — $(1665 \times 0.947 + 259 \times 0.545 + 76 \times 0.629)/2000 = 0.883$, against the macro $(0.947 + 0.545 + 0.629)/3 = 0.707$. The gap, $0.883 - 0.707 = 0.18$, is itself the finding: the small classes are handled much worse than the big one. And the weighted average of recall is always the accuracy, 0.889 here, because support times recall is the number of drives of that class classified correctly.

Read the off-diagonal cells of the matrix, not the accuracy. Degraded is the hard class, and understandably so: it sits between two neighbours that both look like it from a distance.

And the macro–weighted gap is now stark: macro is $(0.947 + 0.545 + 0.629)/3 = 0.707$, weighted is $(1665 \times 0.947 + 259 \times 0.545 + 76 \times 0.629)/2000 = 0.883$. Which to quote is a question about the application: **if missing a failure matters as much as missing a healthy drive, macro is the honest one.**

---

11. Summary

- A linear model on a 0/1 label predicts values that are not probabilities. The **sigmoid** fixes it by modelling the **log-odds** linearly, and e^{w_j} is a multiplier on the **odds** — not on the probability.
- **Cross-entropy is not a design choice**; it is what maximum likelihood produces. Its gradient, $(\hat{y} - y)x$, is identical to linear regression's.
- **Squared error fails** because its gradient carries a factor $\sigma'(z)$ that vanishes exactly where the model is most confidently wrong, and because it is not convex through the sigmoid.
- **96.20% accuracy, zero failures caught.** On an imbalanced problem accuracy is dominated by the majority class: read it against the baseline $1 - \pi$, here 1924/2000.
- The **confusion matrix** is the honest object. **Precision is about the alarms, recall is about the failures.** F_1 uses the harmonic mean so a perfect score on one axis cannot buy a pass on the other.
- The **threshold is a decision**, and the cost-optimal one is $t^* = C_{FP}/(C_{FP} + C_{FN})$ — a function of the costs alone, valid when the probabilities are calibrated. Choosing it on purpose halved the cost here.
- **AUC is the probability a random positive outranks a random negative.** It measures ranking, is blind to calibration, and **hides imbalance** — report the precision-recall curve beside it.
- **Class weights are the threshold by another name**: weights in the ratio k act approximately like a threshold of $1/(1 + k)$. Note how little the AUC moved.
- **With more classes**, softmax turns one score per class into probabilities, the largest wins, and per-class metrics are averaged macro or weighted.

Homework

`Exercises/04_classification_and_metrics.md`, discussed at the start of Lesson 6, **Friday 13 November 2026**.

Notation used in this lesson

Symbol	Meaning
x, X	one drive's telemetry; the design matrix
y, $\hat{y}$	true label (0 or 1); predicted probability
$p^{(i)}$	the model's probability that drive i fails: the same number as $\hat{y}^{(i)}$, written p in the derivations
z	the log-odds, $w^\top x + b$
w, b	coefficients, intercept
m, n	number of examples, number of features
J, L	the cost over the dataset; the loss on one example
H	the matrix of second derivatives of J
σ	the sigmoid
π	the positive rate in the data
t^*	the cost-optimal threshold
C_{FP}, C_{FN}	the cost of a false positive, of a false negative